

Data Analytics Using Tools Practical Programs — Practical 1 to Practical 28

With Full Code & Output • Comments highlighted in Green

Topics: Pandas • NumPy • Matplotlib • Seaborn • Scikit-learn • SQLite • JSON

Practical 1 — Data Analytics Fundamentals – Student Performance Analysis & Prediction

■ Program Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create dataset
data = {
    'Name': ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"],
    'Study_Hours': [2, 3, 4, 5, 6, 7, 8, 1, 9, 10],
    'Attendance': [60, 65, 70, 75, 80, 85, 90, 55, 95, 98],
    'Assignments': [50, 55, 60, 65, 70, 75, 80, 45, 85, 90],
    'Internal_Marks': [40, 45, 50, 55, 60, 65, 70, 35, 75, 80],
    'Final_Marks': [45, 50, 55, 60, 65, 70, 75, 40, 80, 85]
}

df = pd.DataFrame(data)
# Add SrNo
df.insert(0, "SrNo", range(1, len(df)+1))

# Set SrNo as index
df.set_index('SrNo', inplace=True)

# Features & Target
X = df[['Study_Hours', 'Attendance', 'Assignments', 'Internal_Marks']]
y = df['Final_Marks']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Actual Marks:", list(y_test))
print("Predicted Marks:", y_pred)

# Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```

# Add Result column
df['Result'] = df['Final_Marks'].apply(
    lambda x: "Pass" if x >= 40 else "Fail"
)

# Reset index to show SrNo again
df = df.reset_index()

# Arrange columns properly
df = df[['SrNo', 'Name', 'Study_Hours', 'Attendance',
        'Assignments', 'Internal_Marks', 'Final_Marks', 'Result']]

# Final clean table output
print("\nFinal Data (Table Format):\n")
print(df.to_string(index=False))

```

■ Output:

```

Actual Marks: [80, 50]
Predicted Marks: [80. 50.]
Mean Squared Error: 0.0
R2 Score: 1.0

Final Data (Table Format):

SrNo Name Study_Hours Attendance Assignments Internal_Marks Final_Marks Result
1 A 2 60 50 40 45 Pass
2 B 3 65 55 45 50 Pass
3 C 4 70 60 50 55 Pass
4 D 5 75 65 55 60 Pass
5 E 6 80 70 60 65 Pass
6 F 7 85 75 65 70 Pass
7 G 8 90 80 70 75 Pass
8 H 1 55 45 35 40 Pass
9 I 9 95 85 75 80 Pass
10 J 10 98 90 80 85 Pass

```

■ Program Code:

```

import matplotlib.pyplot as plt

# Bar graph: Student vs Final Marks
plt.figure()

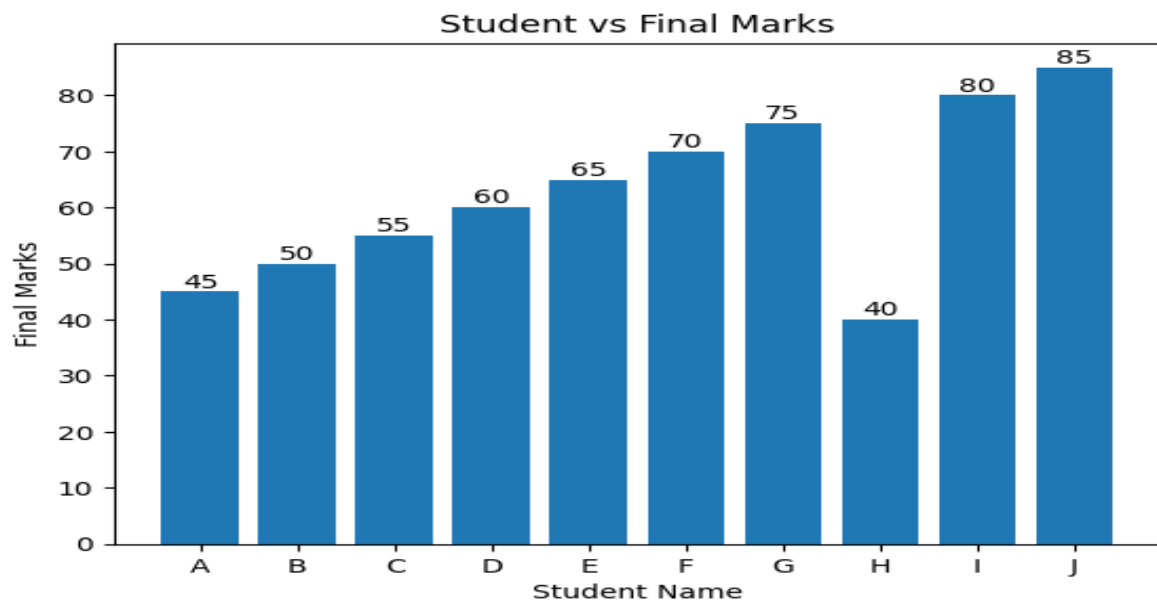
bars = plt.bar(df['Name'], df['Final_Marks'])

# Title and labels
plt.title("Student vs Final Marks")
plt.xlabel("Student Name")
plt.ylabel("Final Marks")

# Show values on bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval,
             str(yval), ha='center', va='bottom')

```

```
plt.show()
```

■ Output:

Practical 2 — Sales Performance Forecasting – Regression & Visualization

■ Program Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Load the Dataset
data = pd.read_csv("Sales_Dataset_2025.csv")

# 1. DESCRIPTIVE ANALYTICS (What happened?)

# This part summarizes the raw data to provide a
# historical view of sales performance.
print("--- Descriptive Analytics ---\n")
print("Total Sales:", data['Sales'].sum())
print("Average Profit:", data['Profit'].mean())

# Visualizing the distribution of sales across months
monthly_sales = data.groupby('Month')['Sales'].sum()
monthly_sales.plot(kind='bar', color='skyblue')
plt.title("Total Sales by Month")
plt.ylabel("Sales ($)")
plt.show()

# 2. DIAGNOSTIC ANALYTICS (Why did it happen?)

print("\n--- Diagnostic Analytics ---\n")

# Select only numeric columns (FIX)
numeric_data = data.select_dtypes(include=['number'])

# Correlation calculation
correlation = numeric_data.corr()

print("Correlation Matrix:\n", correlation)

# Heatmap
sns.heatmap(correlation, annot=True)
plt.title("Correlation Heatmap: Factors Affecting Sales\n")
plt.show()

# 3. PREDICTIVE ANALYTICS (What will happen?)

# This part uses machine learning to forecast
# future sales based on advertising investment.
print("\n--- Predictive Analytics ---\n")
```

```

X = data[['Advertising_Spend']]
y = data['Sales']

# Splitting and Training the Model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

# Evaluation
predictions = model.predict(X_test)
print("Model Accuracy (R2 Score):", r2_score(y_test, predictions))

# Forecasting for a future spend of $6,000
future_spend = [[6000]]
predicted_sales = model.predict(future_spend)
print(f"Predicted Sales for $6,000 Spend: ${predicted_sales[0]:.2f}")

# 4. PRESCRIPTIVE ANALYTICS (What should we do?)

# This part provides recommendations based on the
# data findings to drive business decisions.
print("\n--- Prescriptive Analytics ---")
avg_sales = data['Sales'].mean()

if predicted_sales[0] > avg_sales:
    print("Recommendation: Increase Advertising Budget to maximize revenue growth.")
else:
    print("Recommendation: Optimize Cost Strategy and evaluate marketing channel efficiency.")

```

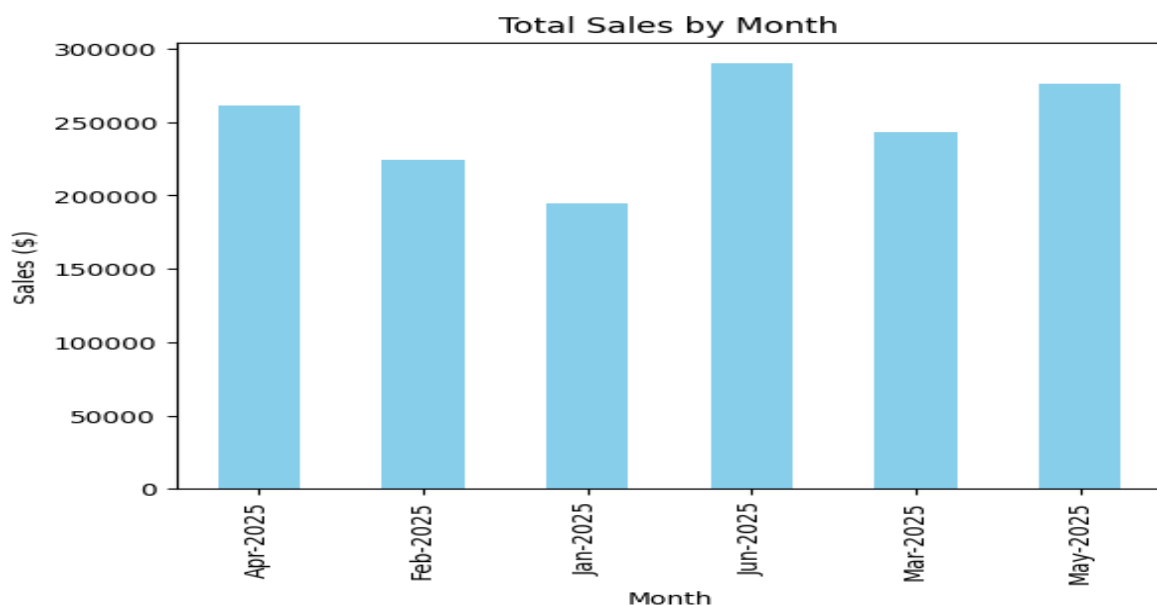
■ Output:

```

--- Descriptive Analytics ---

Total Sales: 1489000
Average Profit: 15916.666666666666

```



```

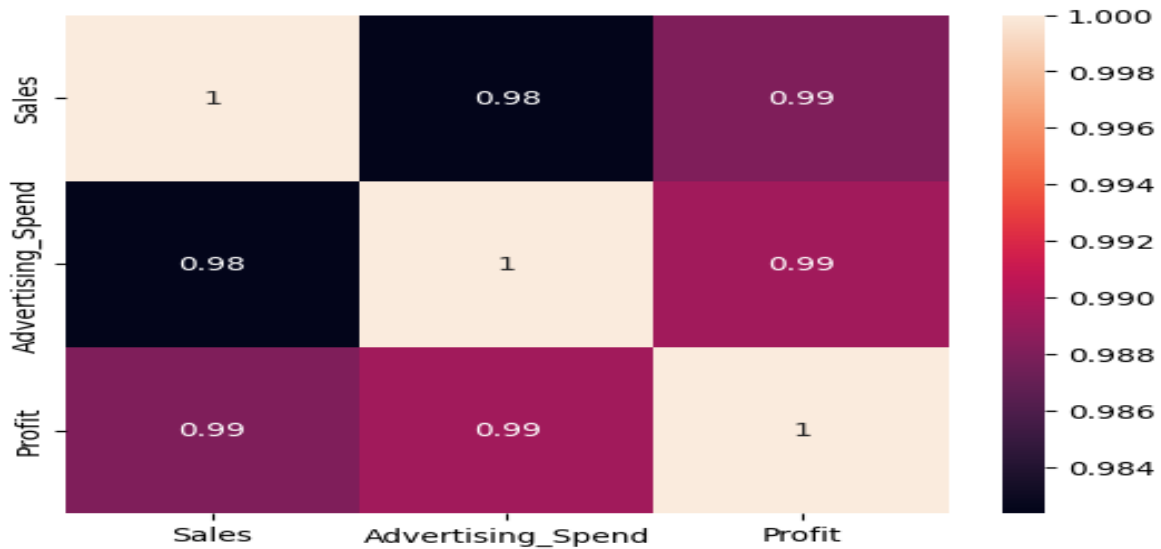
--- Diagnostic Analytics ---

```

Correlation Matrix:

```
          Sales Advertising_Spend Profit
Sales 1.00000 0.982360 0.988060
Advertising_Spend 0.98236 1.000000 0.989395
Profit 0.98806 0.989395 1.000000
```

Correlation Heatmap: Factors Affecting Sales



--- Predictive Analytics ---

```
Model Accuracy (R2 Score): 0.9743575010257353
Predicted Sales for $6,000 Spend: $37513.37
```

--- Prescriptive Analytics ---

Recommendation: Optimize Cost Strategy and evaluate marketing channel efficiency.

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2691: UserWarning: X does not have
valid feature names, but LinearRegression was fitted with feature names
warnings.warn(
```

Practical 3 — Data Suitability and Categorization – Sales Dataset Analysis

■ Program Code:

```
import pandas as pd

# Step 1: Load the dataset
data = pd.read_csv("Sales_Dataset_2025.csv")
```

■ Output:

(No text output)

■ Program Code:

```
import pandas as pd

# Step 1: Load the dataset
data = pd.read_csv("Sales_Dataset_2025.csv")

# Step 2: Display basic information
print("----- BASIC INFORMATION -----")
print("Number of Rows:", data.shape[0])
print("Number of Columns:", data.shape[1])
print("\nColumn Data Types:\n")
print(data.dtypes)

# Step 3: Categorize based on Organization
print("\n----- ORGANIZATION TYPE -----")
if isinstance(data, pd.DataFrame):
    print("The dataset is Structured Data (Rows and Columns format).")

# Step 4: Categorize based on Format
print("\n----- FORMAT TYPE -----")
print("The dataset is in CSV (Comma Separated Values) format.")

# Step 5: Categorize based on Structure (Quantitative & Qualitative)

quantitative = data.select_dtypes(include=['int64', 'float64']).columns
qualitative = data.select_dtypes(include=['object']).columns

print("\n----- STRUCTURE TYPE -----")
print("Quantitative Columns:")
print(list(quantitative))

print("\nQualitative Columns:")
print(list(qualitative))

# Step 6: Analytical Suitability
print("\n----- ANALYTICAL SUITABILITY -----")
print("✓ Suitable for Descriptive Analysis")
print("✓ Suitable for Trend Analysis")
print("✓ Suitable for Statistical Modeling")
print("✓ Suitable for Predictive Analytics")
```

■ Output:

```
----- BASIC INFORMATION -----  
Number of Rows: 24  
Number of Columns: 5  
  
Column Data Types:  
  
Month str  
Region str  
Sales int64  
Advertising_Spend int64  
Profit int64  
dtype: object  
  
----- ORGANIZATION TYPE -----  
The dataset is Structured Data (Rows and Columns format).  
  
----- FORMAT TYPE -----  
The dataset is in CSV (Comma Separated Values) format.  
  
----- STRUCTURE TYPE -----  
Quantitative Columns:  
['Sales', 'Advertising_Spend', 'Profit']  
  
Qualitative Columns:  
['Month', 'Region']  
  
----- ANALYTICAL SUITABILITY -----  
✓ Suitable for Descriptive Analysis  
✓ Suitable for Trend Analysis  
✓ Suitable for Statistical Modeling  
✓ Suitable for Predictive Analytics
```

```
/tmp/ipykernel_662/2451612481.py:25: Pandas4Warning: For backward compatibility, 'str' dtypes are  
included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be  
removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove  
them and silence this warning.  
See https://pandas.pydata.org/docs/user\_guide/migration-3-strings.html#string-migration-select-dtypes  
for details on how to write code that works with pandas 2 and 3.  
    qualitative = data.select_dtypes(include=['object']).columns
```

Practical 4 — Data Sources and Repositories – CSV, Excel, JSON, SQLite

■ Program Code:

```
# Identification of Repositories and Systems that Generate
# or Provide Access to Data for Analysis
# Dataset: Sales_Dataset_2025.csv

# STEP 1: Import Required Libraries
import pandas as pd
import requests
import sqlite3

# STEP 2: Access Data from CSV File (Local Repository)

# Load the dataset from local system
df_csv = pd.read_csv("Sales_Dataset_2025.csv")

print("----- CSV Repository Access -----")

# Display first few rows in console
print(df_csv.head())
```

■ Output:

```
----- CSV Repository Access -----
      Month Region Sales Advertising_Spend Profit
0 Jan-2025 North  52000      8000    12000
1 Jan-2025 South  45000      7000    10000
2 Jan-2025 East   48000      7500    11000
3 Jan-2025 West   50000      8200    11500
4 Feb-2025 North  60000      9000    15000
```

■ Program Code:

```
# STEP 3: Identify Source Information of CSV Dataset

print("\n----- Repository Identification -----")

# Identify the repository type
print("Data Source: Local CSV File Repository")

# Display dataset shape (rows and columns)
print("Dataset Shape:", df_csv.shape)

# Display column names
print("\nDataset Columns:")
print(list(df_csv.columns))

# Display dataset information
print("\nDataset Info:")
df_csv.info()
```

■ Output:

```

----- Repository Identification -----
Data Source: Local CSV File Repository
Dataset Shape: (24, 5)

Dataset Columns:
['Month', 'Region', 'Sales', 'Advertising_Spend', 'Profit']

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 5 columns):
 # Column Non-Null Count Dtype
---  ---
0 Month 24 non-null str
1 Region 24 non-null str
2 Sales 24 non-null int64
3 Advertising_Spend 24 non-null int64
4 Profit 24 non-null int64
dtypes: int64(3), str(2)
memory usage: 1.4 KB

```

■ Program Code:

```

# STEP 4: Identify Data Format and Structure

print("\n----- Data Format and Structure -----")

# Check data types of each column
print(df_csv.dtypes)

# Explain structure
print("\nData Structure: Structured Data (rows and columns)")

```

■ Output:

```

----- Data Format and Structure -----
Month str
Region str
Sales int64
Advertising_Spend int64
Profit int64
dtype: object

Data Structure: Structured Data (rows and columns)

```

■ Program Code:

```

# STEP 5: Demonstrate Excel Repository Access (Example)

print("\n----- Example: Excel Repository -----")

# Convert dataset to Excel (for demonstration)

```

```
df_csv.to_excel("Sales_Data_2025.xlsx", index=False)

# Read Excel file
df_excel = pd.read_excel("Sales_Data_2025.xlsx")

print(df_excel.head())
```

■ Output:

```
----- Example: Excel Repository -----

      Month Region Sales Advertising_Spend Profit
0 Jan-2025 North 52000 8000 12000
1 Jan-2025 South 45000 7000 10000
2 Jan-2025 East 48000 7500 11000
3 Jan-2025 West 50000 8200 11500
4 Feb-2025 North 60000 9000 15000
```

■ Program Code:

```
# STEP 6: Demonstrate JSON Repository Access

print("\n----- Example: JSON Repository -----")

# Convert dataset to JSON
df_csv.to_json("Sales_Data_2025.json")

# Load JSON dataset
df_json = pd.read_json("Sales_Data_2025.json")

print(df_json.head())
```

■ Output:

```
----- Example: JSON Repository -----

      Month Region Sales Advertising_Spend Profit
0 Jan-2025 North 52000 8000 12000
1 Jan-2025 South 45000 7000 10000
2 Jan-2025 East 48000 7500 11000
3 Jan-2025 West 50000 8200 11500
4 Feb-2025 North 60000 9000 15000
```

■ Program Code:

```
# STEP 8: Demonstrate Database Repository Access

print("\n----- Example: Database Repository (SQLite) -----")

# Create SQLite connection
conn = sqlite3.connect("Sales_Data_2025.db")

# Store dataset into database table
df_csv.to_sql("Sales", conn, if_exists="replace", index=False)

# Read data from SQL table
```

```
df_sql = pd.read_sql("SELECT * FROM Sales", conn)

print(df_sql.head())
```

■ Output:

```
----- Example: Database Repository (SQLite) -----
      Month Region Sales Advertising_Spend Profit
0 Jan-2025 North 52000 8000 12000
1 Jan-2025 South 45000 7000 10000
2 Jan-2025 East 48000 7500 11000
3 Jan-2025 West 50000 8200 11500
4 Feb-2025 North 60000 9000 15000
```

■ Program Code:

```
# STEP 9: Demonstrate Online Data Repository Access (Sales Data)

import pandas as pd

print("\n----- Example: Online Sales Data Repository -----")

try:
    # Working Sales Dataset URL
    url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"

    # Load dataset from URL
    df_api = pd.read_csv(url)

    print("Data fetched successfully!\n")

    # Display first 5 records
    print("Sales-like Data (First 5 Records):\n")
    print(df_api.head())

    # Dataset shape
    print("\nDataset Shape:", df_api.shape)

    # Column names
    print("\nColumns:", list(df_api.columns))

except Exception as e:
    print("Error occurred while accessing online dataset:", e)
```

■ Output:

```
----- Example: Online Sales Data Repository -----
Error occurred while accessing online dataset: HTTP Error 403: Forbidden
```

■ Program Code:

```
# STEP 10: Compare Data Types from Different Repositories

print("\n----- Data Type Comparison -----")
```

```
print("\nCSV Data Types:\n")
print(df_csv.dtypes)

print("\nJSON Data Types:\n")
print(df_json.dtypes)

print("\nSQL Data Types:\n")
print(df_sql.dtypes)

# STEP 11: Final Conclusion

print("\n----- Conclusion -----")

print("1. The dataset 'Sales_Dataset_2025.csv' is stored in a Local CSV Repository.")
print("2. The data is structured in tabular format with rows and columns.")
print("3. The dataset contains important business features such as Month, Region, Sales, Advertising_Spend, and Profit.")
print("4. Sales and Advertising_Spend are quantitative variables used for analysis and prediction.")
print("5. Month and Region are qualitative variables used for categorization and grouping.")
print("6. This type of data is commonly generated by business systems like Sales Management Systems, CRM, and Marketing Platforms.")
print("7. The dataset is highly suitable for data analysis, visualization, and machine learning models.")
```

■ Output:

```
----- Data Type Comparison -----
```

```
CSV Data Types:
```

```
Month str
Region str
Sales int64
Advertising_Spend int64
Profit int64
dtype: object
```

```
JSON Data Types:
```

```
Month str
Region str
Sales int64
Advertising_Spend int64
Profit int64
dtype: object
```

```
SQL Data Types:
```

```
Month str
Region str
Sales int64
```

```
Advertising_Spend int64
Profit int64
dtype: object
```

----- Conclusion -----

1. The dataset 'Sales_Dataset_2025.csv' is stored in a Local CSV Repository.
2. The data is structured in tabular format with rows and columns.
3. The dataset contains important business features such as Month, Region, Sales, Advertising_Spend, and Profit.
4. Sales and Advertising_Spend are quantitative variables used for analysis and prediction.
5. Month and Region are qualitative variables used for categorization and grouping.
6. This type of data is commonly generated by business systems like Sales Management Systems, CRM, and Marketing Platforms.
7. The dataset is highly suitable for data analysis, visualization, and machine learning models.

Practical 5 — Predictive Modeling using Regression – Sales Dataset

■ Program Code:

```
# STEP 1: IMPORT LIBRARIES

# Libraries are tools that help us perform data analysis
# and machine learning tasks.

import pandas as pd # Used for working with datasets (tables)
import numpy as np # Used for numerical calculations
import matplotlib.pyplot as plt # Used for creating graphs

# Machine learning tools from scikit-learn
from sklearn.model_selection import train_test_split # Splits dataset into training and testing
from sklearn.preprocessing import LabelEncoder # Converts text data into numbers
from sklearn.linear_model import LinearRegression # Machine learning model used for prediction
from sklearn.metrics import mean_squared_error, r2_score # Used to evaluate model performance
```

■ Output:

(No text output)

■ Program Code:

```
# STEP 2: LOAD DATASET

# Read the student performance dataset from CSV file.

data = pd.read_csv("Sales_Dataset_2025.csv")

# Display first 5 rows to understand the dataset structure
print("First 5 rows of dataset:")
print(data.head().T)
```

■ Output:

```
First 5 rows of dataset:
      0 1 2 3 4
Month Jan-2025 Jan-2025 Jan-2025 Jan-2025 Feb-2025
Region North South East West North
Sales 52000 45000 48000 50000 60000
Advertising_Spend 8000 7000 7500 8200 9000
Profit 12000 10000 11000 11500 15000
```

■ Program Code:

```
# STEP 3: DATA UNDERSTANDING

# Check basic information about the dataset.

# Check number of rows and columns
print("Dataset shape:")
print(data.shape)

# Check column names and data types
print("\nDataset information:")
print(data.info())
```

```
# Statistical summary of numeric columns
print("\nStatistical summary:")
print(data.describe())

# Check missing values in each column
print("\nMissing values in dataset:")
print(data.isnull().sum())
```

■ Output:

```
Dataset shape:
(24, 5)

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 24 entries, 0 to 23
Data columns (total 5 columns):
 # Column Non-Null Count Dtype
---  ---
 0 Month 24 non-null str
 1 Region 24 non-null str
 2 Sales 24 non-null int64
 3 Advertising_Spend 24 non-null int64
 4 Profit 24 non-null int64
dtypes: int64(3), str(2)
memory usage: 1.4 KB
None

Statistical summary:
           Sales Advertising_Spend Profit
count 24.000000 24.000000 24.000000
mean 62041.666667 9100.000000 15916.666667
std 8685.215657 1072.177792 3377.246971
min 45000.000000 7000.000000 10000.000000
25% 56250.000000 8200.000000 13375.000000
50% 62500.000000 9100.000000 15750.000000
75% 69250.000000 9850.000000 18250.000000
max 75000.000000 11000.000000 22000.000000

Missing values in dataset:
Month 0
Region 0
Sales 0
Advertising_Spend 0
Profit 0
dtype: int64
```

■ Program Code:

```
# STEP 4: DATA VISUALIZATION

# 1. Sales Distribution
plt.figure()
plt.hist(data['Sales'], bins=10)
plt.title("Sales Distribution")
```

```
plt.xlabel("Sales Amount")
plt.ylabel("Frequency")
plt.show()

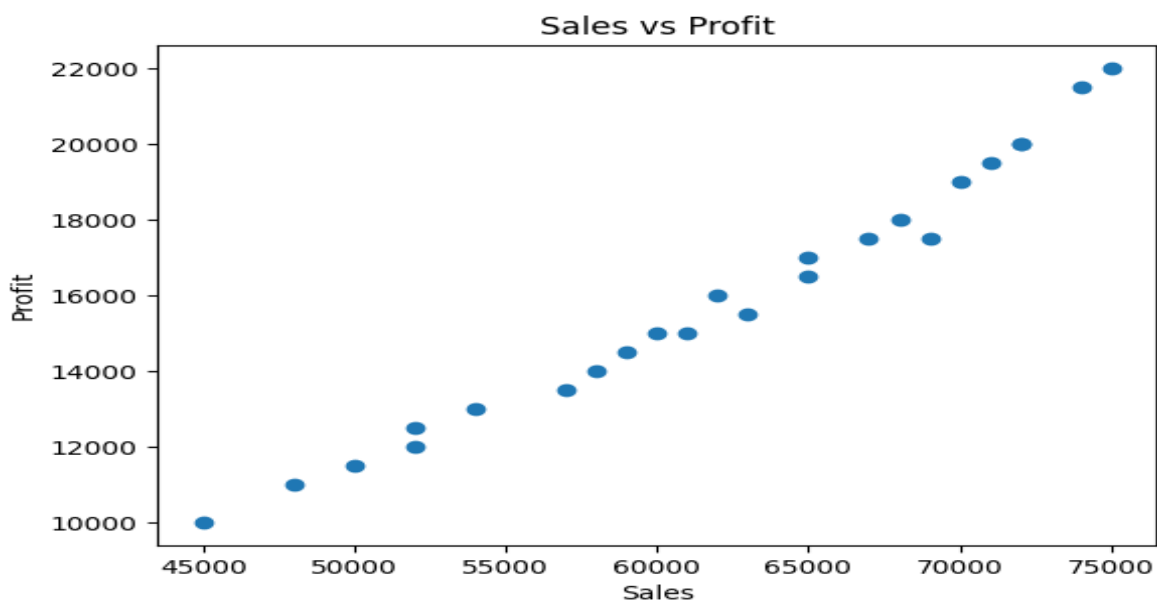
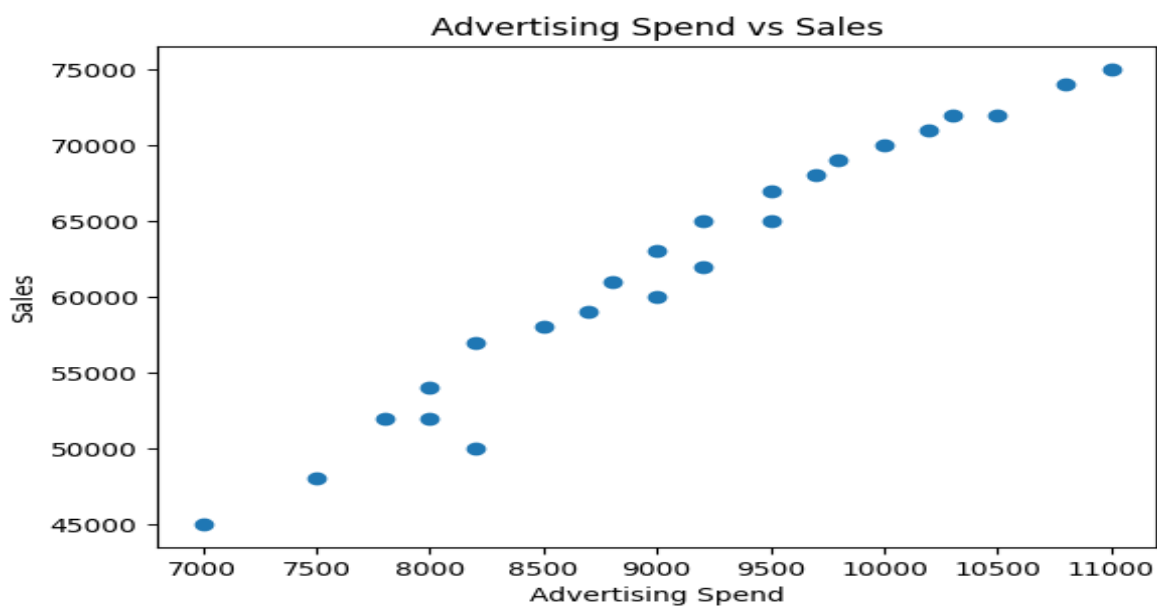
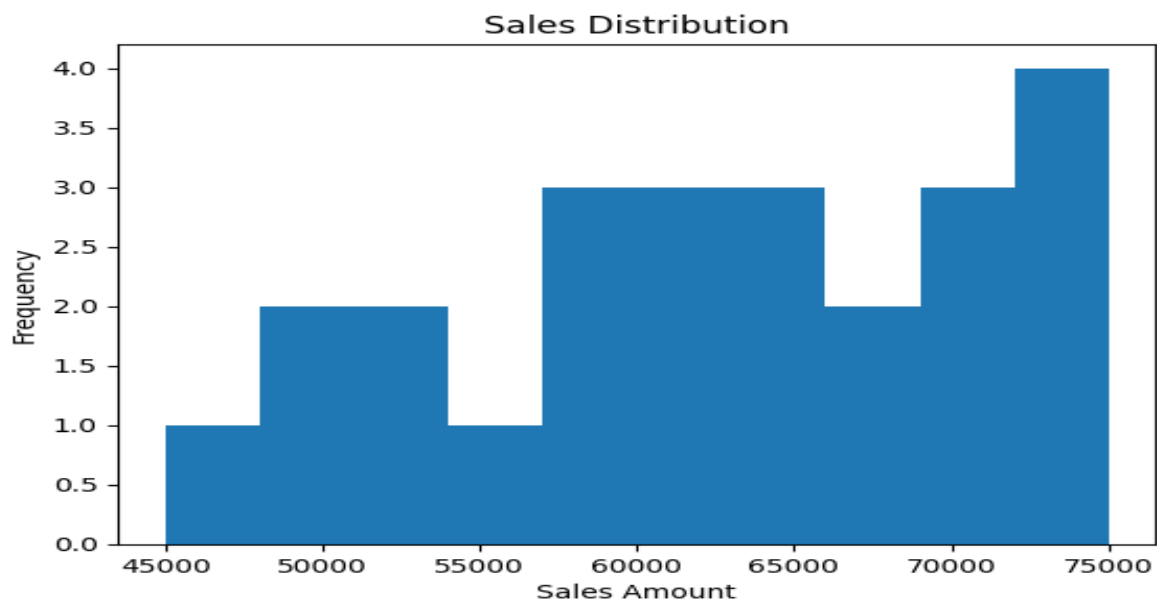
# 2. Advertising Spend vs Sales
plt.figure()
plt.scatter(data['Advertising_Spend'], data['Sales'])
plt.title("Advertising Spend vs Sales")
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.show()

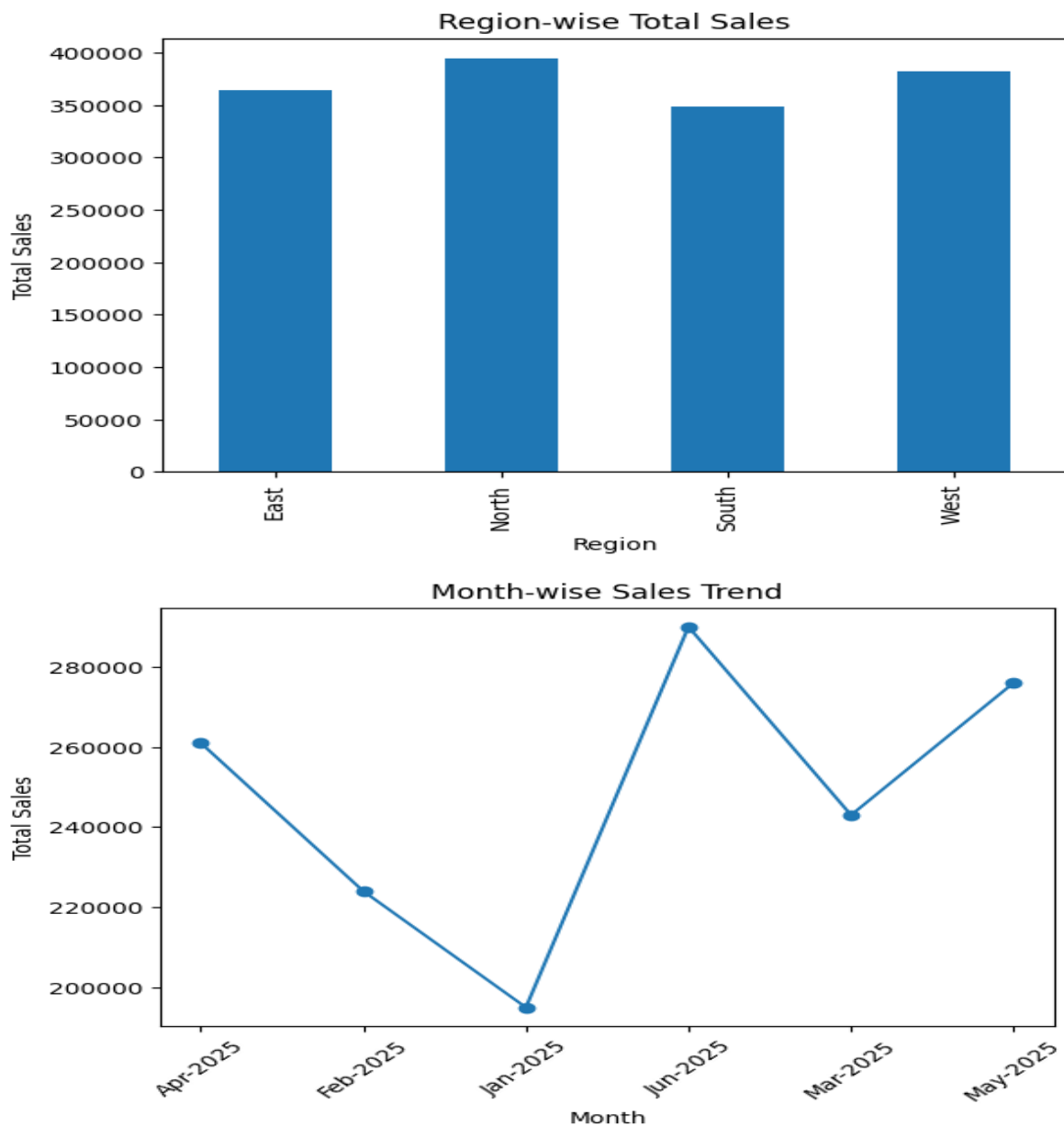
# 3. Sales vs Profit
plt.figure()
plt.scatter(data['Sales'], data['Profit'])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.show()

# 4. Region-wise Total Sales (Bar Chart)
plt.figure()
data.groupby('Region')['Sales'].sum().plot(kind='bar')
plt.title("Region-wise Total Sales")
plt.xlabel("Region")
plt.ylabel("Total Sales")
plt.show()

# 5. Month-wise Sales Trend (VERY IMPORTANT ■)
plt.figure()
data.groupby('Month')['Sales'].sum().plot(kind='line', marker='o')
plt.title("Month-wise Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.show()
```

■ Output:





■ Program Code:

```
# STEP 5: DATA PREPARATION (Sales Dataset 2025)

# Create encoder object
encoder = LabelEncoder()

# Convert categorical columns into numeric
data['Month'] = encoder.fit_transform(data['Month'])
data['Region'] = encoder.fit_transform(data['Region'])

# Display updated dataset
print("Dataset after converting categorical values:\n")
print(data.head(7))
```

■ Output:

```
Dataset after converting categorical values:
```

```

    Month Region Sales Advertising_Spend Profit
0 2 1 52000 8000 12000
1 2 2 45000 7000 10000
2 2 0 48000 7500 11000
3 2 3 50000 8200 11500
4 1 1 60000 9000 15000
5 1 2 52000 7800 12500
6 1 0 54000 8000 13000

```

■ Program Code:

```

# STEP 6: REMOVE UNNECESSARY COLUMN

# In this dataset, no unnecessary column like ID is present
# So we skip this step

print("Columns in dataset:")
print(data.columns)


# STEP 7: DEFINE FEATURES AND TARGET

# X = input features
X = data.drop('Profit', axis=1)

# y = target variable (Profit)
y = data['Profit']


# STEP 8: TRAIN-TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)


# STEP 9: MODEL TRAINING

model = LinearRegression()

```

```
# Train model
model.fit(X_train, y_train)

print("Model training completed.")

# STEP 10: PREDICT RESULTS

y_pred = model.predict(X_test)

print("\nSample Predicted Profit Values:")
print(y_pred[:10])

# STEP 11: MODEL EVALUATION

from sklearn.metrics import mean_squared_error, r2_score

# Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# R2 Score
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error:", mse)
print("R2 Score:", r2)
```

■ Output:

```
Columns in dataset:
Index(['Month', 'Region', 'Sales', 'Advertising_Spend', 'Profit'], dtype='str')
Training data size: (19, 4)
Testing data size: (5, 4)
Model training completed.

Sample Predicted Profit Values:
[17210.83433689 20179.43274364 12332.19582664 18129.24471836
 16087.78153658]

Mean Squared Error: 42282.21794523038
R2 Score: 0.9939940031327797
```

Practical 6 — Student Performance Analysis – Data Storage with HTML Output

■ Program Code:

```
# DATA STORAGE WITH HTML TABLE OUTPUT (DISPLAY)

import pandas as pd
from IPython.display import display, HTML

# STEP 1: Load dataset
data = pd.read_csv("student_performance_data.csv")

# STEP 2: Display First 5 Rows
print("First 5 Rows:")
display(HTML(data.head().to_html()))

# STEP 3: Display Dataset Shape
print("\nDataset Shape:")
shape_df = pd.DataFrame({
    "Rows": [data.shape[0]],
    "Columns": [data.shape[1]]
})
display(HTML(shape_df.to_html(index=False)))

# STEP 4: Display Dataset Info
print("\nDataset Info:")
info_df = pd.DataFrame({
    "Column": data.columns,
    "Non-Null Count": data.count().values,
    "Data Type": data.dtypes.values
})
display(HTML(info_df.to_html(index=False)))

# STEP 5: Save dataset copy
data.to_csv("student_data_storage_copy.csv", index=False)

print("\nDataset successfully stored and accessed.")
```

■ Output:

```
First 5 Rows:
<IPython.core.display.HTML object>

Dataset Shape:
<IPython.core.display.HTML object>

Dataset Info:
<IPython.core.display.HTML object>

Dataset successfully stored and accessed.
```

■ Program Code:

```
# DATA INTEGRATION WITH HTML TABLE OUTPUT (DISPLAY)
```

```

import pandas as pd
from IPython.display import display, HTML

# STEP 1: Load dataset
data = pd.read_csv("student_performance_data.csv")

# STEP 2: Create two datasets
academic_data = data[['StudentID', 'GPA', 'Major']]
activity_data = data[['StudentID', 'ExtraCurricularActivities', 'PartTimeJob']]

# STEP 3: Display Academic Data
print("Academic Data Sample:")
display(HTML(academic_data.head().to_html()))

# STEP 4: Display Activity Data
print("\nActivity Data Sample:")
display(HTML(activity_data.head().to_html()))

# STEP 5: Merge datasets
integrated_data = pd.merge(
    academic_data,
    activity_data,
    on="StudentID"
)

# STEP 6: Display Integrated Data
print("\nIntegrated Dataset:")
display(HTML(integrated_data.head().to_html()))

```

■ Output:

```

Academic Data Sample:
<IPython.core.display.HTML object>

Activity Data Sample:
<IPython.core.display.HTML object>

Integrated Dataset:
<IPython.core.display.HTML object>

```

■ Program Code:

```

# PATTERN DISCOVERY WITH HTML TABLE OUTPUT (DISPLAY)

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML

# STEP 1: Load dataset
data = pd.read_csv("student_performance_data.csv")

```

```
# STEP 2: Statistical Summary (HTML Table)
print("Statistical Summary:")
display(HTML(data.describe().to_html()))

# STEP 3: GPA Distribution
plt.hist(data['GPA'], bins=10)
plt.title("GPA Distribution")
plt.xlabel("GPA")
plt.ylabel("Number of Students")
plt.show()

# STEP 4: Study Hours vs GPA
plt.scatter(data['StudyHoursPerWeek'], data['GPA'])
plt.title("Study Hours vs GPA")
plt.xlabel("Study Hours Per Week")
plt.ylabel("GPA")
plt.show()

# STEP 5: Attendance vs GPA
plt.scatter(data['AttendanceRate'], data['GPA'])
plt.title("Attendance Rate vs GPA")
plt.xlabel("Attendance Rate")
plt.ylabel("GPA")
plt.show()

# STEP 6: Correlation Matrix (FIXED for all pandas versions)
numeric_data = data.select_dtypes(include=['number'])
correlation_matrix = numeric_data.corr()

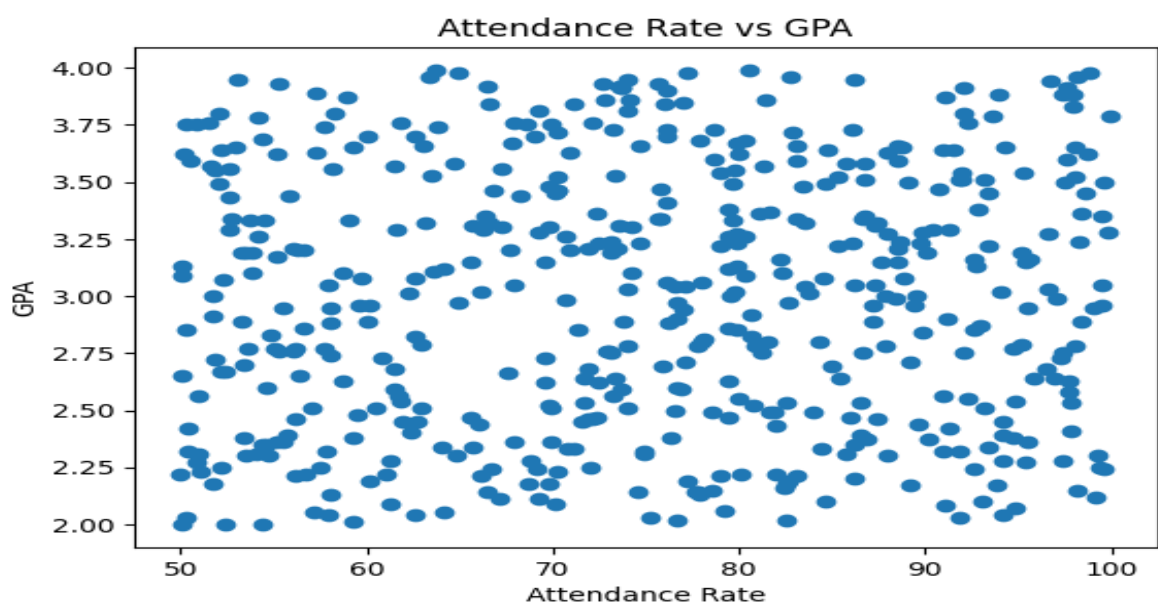
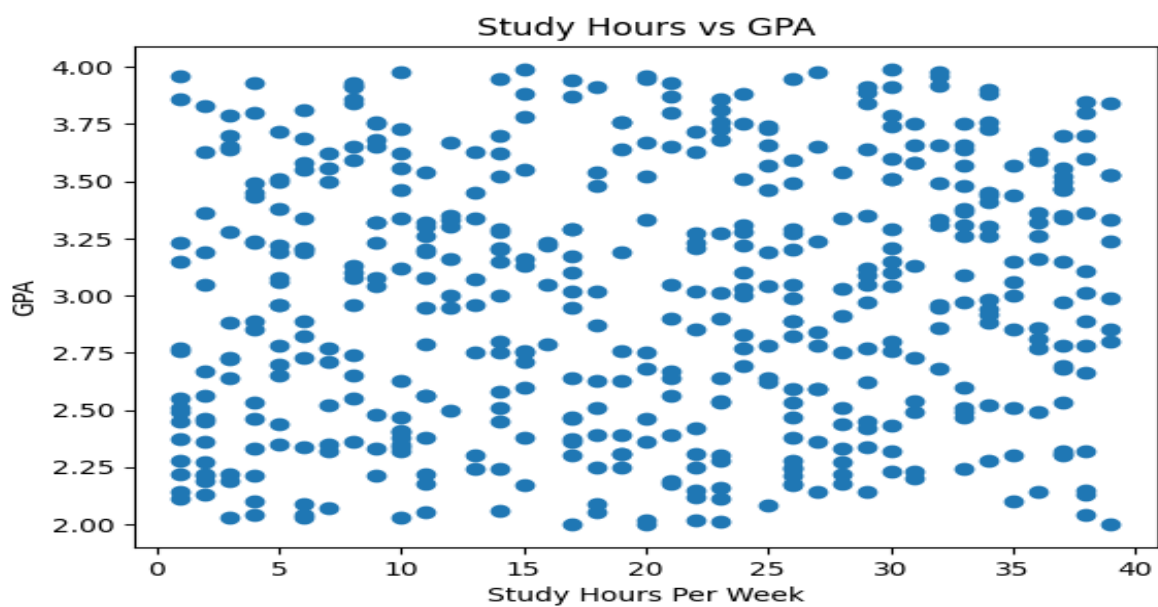
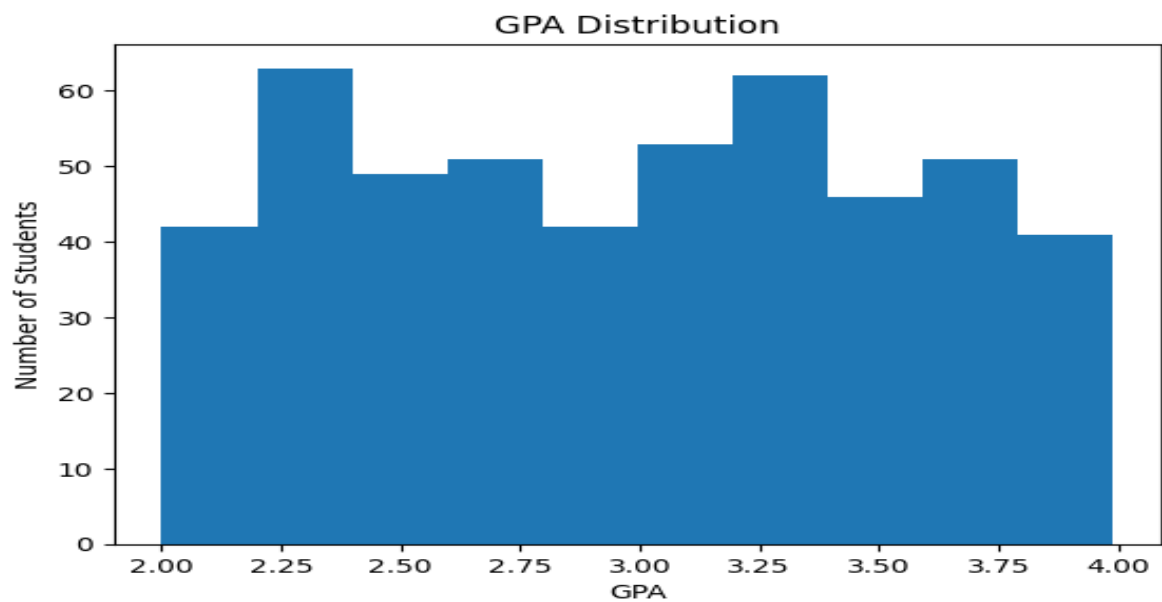
print("\nCorrelation Matrix:")
display(HTML(correlation_matrix.to_html()))

# STEP 7: Heatmap Visualization
sns.heatmap(correlation_matrix, annot=True)
plt.title("Feature Correlation Heatmap\n")
plt.show()
```

■ Output:

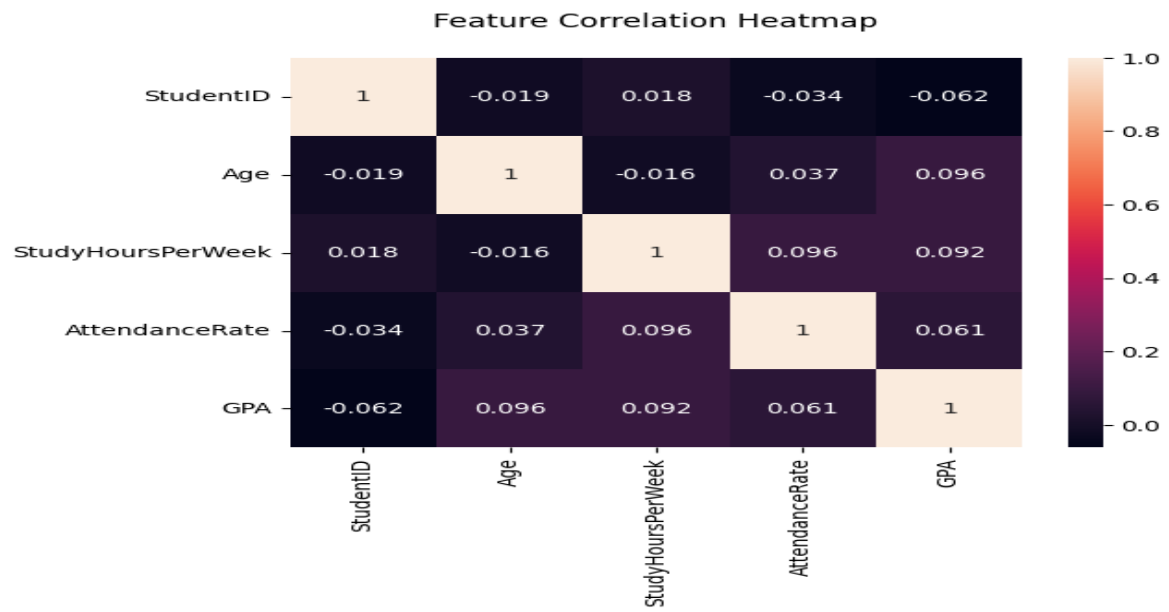
```
Statistical Summary:
```

```
<IPython.core.display.HTML object>
```



Correlation Matrix:

<IPython.core.display.HTML object>



Practical 7 — Student Performance Analytics Dashboard – Data Access Control

■ Program Code:

```
# DATA ACCESS CONTROL WITH HTML TABLE DISPLAY

import pandas as pd
from IPython.display import display, HTML

# STEP 1: Load dataset
data = pd.read_csv("student_performance_data.csv")

# STEP 2: Define allowed columns
allowed_columns = ['StudentID', 'Gender', 'Age', 'Major']

# STEP 3: Create restricted dataset
restricted_data = data[allowed_columns]

# STEP 4: Display restricted dataset in table format
print("Restricted Dataset (Authorized View):")
display(HTML(restricted_data.head().to_html()))
```

■ Output:

```
Restricted Dataset (Authorized View):
<IPython.core.display.HTML object>
```

■ Program Code:

```
# DATA PRIVACY PROTECTION WITH HTML TABLE DISPLAY

import pandas as pd
from IPython.display import display, HTML

# Load dataset
data = pd.read_csv("student_performance_data.csv")

# STEP 1: Create copy (protect original data)
privacy_data = data.copy()

# STEP 2: Mask StudentID
privacy_data['StudentID'] = privacy_data['StudentID'].apply(
    lambda x: "ID_****" + str(x)[-3:]
)

# STEP 3: Display anonymized dataset in table format
print("Anonymized Dataset:")
display(HTML(privacy_data.head().to_html()))
```

■ Output:

```
Anonymized Dataset:
<IPython.core.display.HTML object>
```

■ Program Code:

```
# PART 3: DATA SECURITY
# Security practices protect datasets from data loss or unauthorized changes
```

```
# STEP 9: Create a secure backup of the dataset
# to_csv() saves the dataset to another file
#--CODE data.to_csv("student_backup.csv", index=False)

# STEP 10: Display confirmation message
print("Secure dataset backup created successfully")
```

■ Output:

```
Secure dataset backup created successfully
```

■ Program Code:

```
# DATA QUALITY STANDARDS WITH HTML TABLE DISPLAY

import pandas as pd
from IPython.display import display, HTML

# Load dataset
data = pd.read_csv("student_performance_data.csv")

# STEP 1: Check missing values
missing_values = data.isnull().sum()

# Convert to table format
missing_df = missing_values.reset_index()
missing_df.columns = ["Column Name", "Missing Values"]

print("Missing Values in Dataset:")
display(HTML(missing_df.to_html(index=False)))

# STEP 2: Check duplicate records
duplicate_records = data.duplicated().sum()

print("\nDuplicate Records:")
display(HTML(duplicate_df.to_html(index=False)))
```

■ Output:

```
Missing Values in Dataset:
```

```
<IPython.core.display.HTML object>
```

```
Duplicate Records:
```

```
NameError: name 'duplicate_df' is not defined
```

■ Program Code:

```
# PART 5: DATA COMPLIANCE MONITORING
# This step checks if the dataset follows defined governance policies

# STEP 13: Identify invalid attendance values
# The rule is: AttendanceRate must be between 0 and 100
# This line finds rows where AttendanceRate is less than 0 or greater than 100
invalid_attendance = data[(data['AttendanceRate'] < 0) | (data['AttendanceRate'] > 100)]

# STEP 14: Display records that violate the rule
# If no rows are returned, it means the dataset follows the policy
```

```
print("Invalid Attendance Records:")  
print(invalid_attendance)
```

■ Output:

```
Invalid Attendance Records:  
Empty DataFrame  
Columns: [StudentID, Gender, Age, StudyHoursPerWeek, AttendanceRate, GPA, Major, PartTimeJob,  
ExtraCurricularActivities]  
Index: []
```

Practical 8 — Data Analytics Workflow System – End-to-End Pipeline

■ Program Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import HTML
```

■ Output:

(No text output)

■ Program Code:

```
data = pd.read_csv("student_scores_dataset.csv")
data.head()
html_table = data.to_html(index=False, justify='center')

# Step 6: Display in notebook
display(HTML(html_table))
```

■ Output:

<IPython.core.display.HTML object>

■ Program Code:

```
data.info()
data.describe()
```

■ Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 6 columns):
 # Column Non-Null Count Dtype
---
 0 Student_ID 50 non-null int64
 1 Name 50 non-null str
 2 Math_Score 50 non-null int64
 3 Science_Score 50 non-null int64
 4 English_Score 50 non-null int64
 5 Attendance 50 non-null int64
dtypes: int64(5), str(1)
memory usage: 2.7 KB
```

	Student_ID	Math_Score	Science_Score	English_Score	Attendance
count	50.000000	50.000000	50.000000	50.000000	50.000000
mean	25.500000	77.060000	78.320000	77.020000	89.220000
std	14.57738	8.634081	7.313504	7.155674	4.599423
min	1.000000	60.000000	64.000000	65.000000	80.000000
25%	13.250000	70.000000	72.250000	71.000000	86.000000
50%	25.500000	77.000000	78.500000	76.500000	89.000000
75%	37.750000	83.750000	84.750000	83.000000	92.750000
max	50.000000	92.000000	91.000000	90.000000	98.000000

■ Program Code:

```
data.isnull().sum()
```

■ **Output:**

```
Student_ID 0
Name 0
Math_Score 0
Science_Score 0
English_Score 0
Attendance 0
dtype: int64
```

■ **Program Code:**

```
data.fillna(data.mean(), inplace=True)
```

■ **Output:**

```
TypeError: Cannot perform reduction 'mean' with string dtype
```

■ **Program Code:**

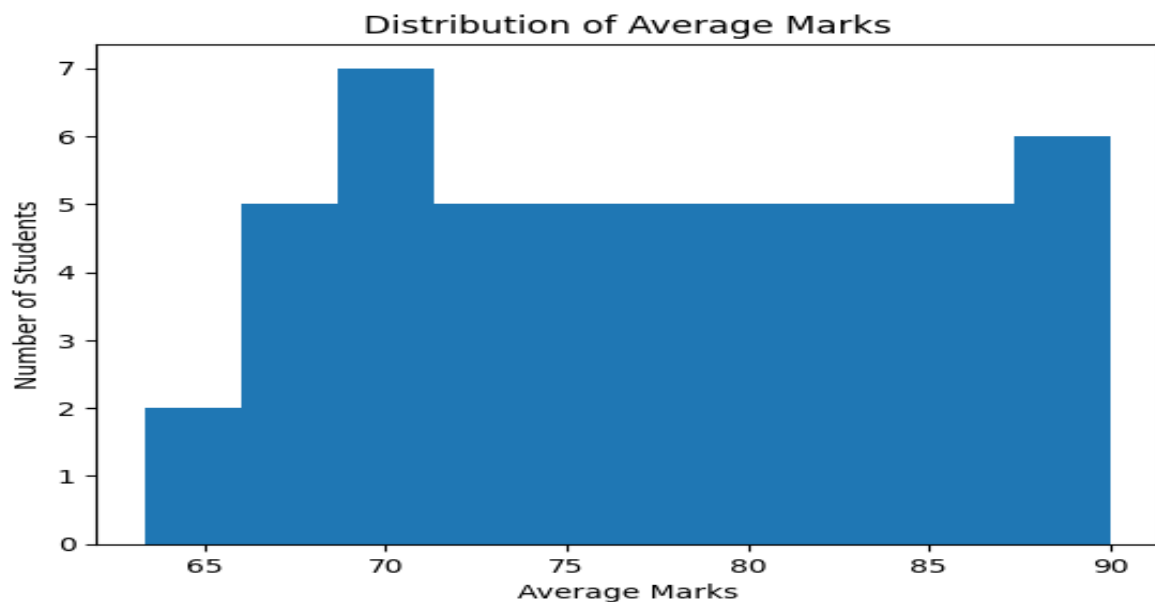
```
data["Average"] = (data["Math_Score"] + data["Science_Score"] + data["English_Score"]) / 3
```

■ **Output:**

```
(No text output)
```

■ **Program Code:**

```
plt.hist(data["Average"])
plt.title("Distribution of Average Marks")
plt.xlabel("Average Marks")
plt.ylabel("Number of Students")
plt.show()
```

■ **Output:**■ **Program Code:**

```
data.sort_values(by="Average", ascending=False).head()
```

■ **Output:**

```
Student_ID Name Math_Score Science_Score English_Score Attendance \
16 17 Aryan 92 90 88 98
```

```
30 31 Pranav 91 89 90 97
8 9 Ishaan 91 88 90 97
49 50 Rakesh 88 90 89 95
42 43 Rishi 90 88 87 96
```

Average

```
16 90.000000
30 90.000000
8 89.666667
49 89.000000
42 88.333333
```

■ Program Code:

```
# Step 5: Convert DataFrame to HTML with center alignment and no index
html_table = data.to_html(index=False, justify='center')

# Step 6: Display in notebook
display(HTML(html_table))
```

■ Output:

```
<IPython.core.display.HTML object>
```

Practical 9 — Pandas Data Integration and Analysis – Student Attendance & Marks

■ Program Code:

```
import pandas as pd
from IPython.display import HTML
```

■ Output:

(No text output)

■ Program Code:

```
marks = pd.read_csv("student_marks.csv")
attendance = pd.read_csv("student_attendance.csv")
marks.head()
attendance.head()

html_table = marks.to_html(index=False, justify='center')

# Step 6: Display in notebook
display(HTML(html_table))
html_table = attendance.to_html(index=False, justify='center')

# Step 6: Display in notebook
display(HTML(html_table))
```

■ Output:

```
<IPython.core.display.HTML object>
<IPython.core.display.HTML object>
```

■ Program Code:

```
marks.info()
attendance.info()
```

■ Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
 # Column Non-Null Count Dtype
---  ---
 0 ID 3 non-null int64
 1 Name 3 non-null str
 2 Math 3 non-null int64
 3 Science 3 non-null int64
dtypes: int64(3), str(1)
memory usage: 243.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 # Column Non-Null Count Dtype
---  ---
 0 ID 3 non-null int64
 1 Attendance 3 non-null int64
dtypes: int64(2)
memory usage: 180.0 bytes
```

■ Program Code:

```
merged_data = pd.merge(marks, attendance, on="ID")
merged_data
```

■ Output:

```
   ID Name Math Science Attendance
0  1  Rahul  80   75    90
1  2  Anita  90   88    85
2  3  Karan  70   65    88
```

■ Program Code:

```
merged_data.to_csv("student_complete_data.csv", index=False)
```

■ Output:

(No text output)

■ Program Code:

```
merged_data["Total"] = merged_data["Math"] + merged_data["Science"]
merged_data
```

■ Output:

```
   ID Name Math Science Attendance Total
0  1  Rahul  80   75    90   155
1  2  Anita  90   88    85   178
2  3  Karan  70   65    88   135
```

Practical 10 — Data Cleaning and Preprocessing – Employee Dataset

■ Program Code:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from IPython.display import HTML
```

■ Output:

(No text output)

■ Program Code:

```
data = pd.read_csv("employee_data.csv")
data.head()

html_table = data.to_html(index=False, justify='center')

display(HTML(html_table))
```

■ Output:

<IPython.core.display.HTML object>

■ Program Code:

```
data["Join_Date"] = pd.to_datetime(data["Join_Date"])
```

■ Output:

ValueError: time data "20-01-2023" doesn't match format "%m-%d-%Y". You might want to try:

- passing `format` if your strings have a consistent format;
- passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily in exactly the same format;
- passing `format='mixed'`, and the format will be inferred for each element individually. You might want to use `dayfirst` alongside this.

■ Program Code:

```
data["Name"] = data["Name"].astype(str)
```

■ Output:

(No text output)

■ Program Code:

```
scaler = StandardScaler()
data["Salary_Scaled"] = scaler.fit_transform(data[["Salary"]])
```

■ Output:

(No text output)

■ Program Code:

```
data.head()
html_table = data.to_html(index=False, justify='center')

display(HTML(html_table))
```

■ Output:

```
<IPython.core.display.HTML object>
```

■ Program Code:

```
data.to_csv("standardized_employee_data.csv", index=False)
```

■ Output:

```
(No text output)
```

■ Program Code:

```
data.head()  
html_table = data.to_html(index=False, justify='center')  
  
display(HTML(html_table))
```

■ Output:

```
<IPython.core.display.HTML object>
```

Practical 11 — Employee Data Analysis – Load, Inspect, Merge, GroupBy

■ Program Code:

```
#Step 1: Load Dataset and Inspect
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats

df = pd.read_csv('employees.csv')

print("First 5 Rows:\n", df.head())
print("\nInfo:\n")
print(df.info())

print("\nSummary:\n")
print(df.describe(include='all'))
```

■ Output:

```
First 5 Rows:
   EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER HIRE_DATE \
0  198  Donald  OConnell DOCONNEL  650.507.9833  21-JUN-07
1  199  Douglas  Grant  DGRANT  650.507.9844  13-JAN-08
2  200  Jennifer Whalen  NaN  515.123.4444  17-SEP-03
3  201  Michael  Hartstein MHARTSTE  515.123.5555  17-FEB-04
4  202    Pat  Fay  PFAY  603.123.6666  17-AUG-05

   JOB_ID SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
0 SH_CLERK 2600.0 - 124 50
1 SH_CLERK  NaN - 124 50
2 AD_ASST 4400.0 - 101 10
3 MK_MAN 13000.0 - 100 20
4 MK_REP  6000.0 - 201 20

Info:

<class 'pandas.DataFrame'>
RangeIndex: 55 entries, 0 to 54
Data columns (total 11 columns):
 # Column Non-Null Count Dtype
---  ---
0  EMPLOYEE_ID 55 non-null int64
1  FIRST_NAME  45 non-null str
2  LAST_NAME   55 non-null str
3  EMAIL       49 non-null str
4  PHONE_NUMBER 55 non-null str
5  HIRE_DATE   55 non-null str
6  JOB_ID      55 non-null str
7  SALARY      50 non-null float64
8  COMMISSION_PCT 55 non-null str
9  MANAGER_ID  55 non-null str
10 DEPARTMENT_ID 55 non-null int64
dtypes: float64(1), int64(2), str(8)
memory usage: 7.6 KB
```

None

Summary:

```

EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER HIRE_DATE \
count 55.000000 45 55 49 55 55
unique NaN 40 50 45 49 47
top NaN Michael Ernst BERNST INVALID 07-JUN-02
freq NaN 2 2 2 6 4
mean 133.400000 NaN NaN NaN NaN NaN
std 32.545467 NaN NaN NaN NaN NaN
min 100.000000 NaN NaN NaN NaN NaN
25% 111.500000 NaN NaN NaN NaN NaN
50% 124.000000 NaN NaN NaN NaN NaN
75% 136.000000 NaN NaN NaN NaN NaN
max 206.000000 NaN NaN NaN NaN NaN

```

```

JOB_ID SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
count 55 50.000000 55 55 55.000000
unique 17 NaN 1 14 NaN
top ST_CLERK NaN - 100 NaN
freq 18 NaN 55 10 NaN
mean NaN 5168.480000 NaN NaN 58.000000
std NaN 5956.400976 NaN NaN 24.675674
min NaN -8200.000000 NaN NaN 10.000000
25% NaN 2525.000000 NaN NaN 50.000000
50% NaN 3900.000000 NaN NaN 50.000000
75% NaN 8200.000000 NaN NaN 60.000000
max NaN 24000.000000 NaN NaN 110.000000

```

■ Program Code:

```

# step 2 : Check missing values count
print("\nMissing Values Count:\n", df.isnull().sum())

missing_percentage = df.isnull().mean() * 100
print("\nMissing %:\n", missing_percentage)

# Heatmap
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap='viridis')
plt.title('Missing Data Heatmap')
plt.show()

```

■ Output:

```

Missing Values Count:
EMPLOYEE_ID 0
FIRST_NAME 10
LAST_NAME 0
EMAIL 6
PHONE_NUMBER 0
HIRE_DATE 0
JOB_ID 0
SALARY 5
COMMISSION_PCT 0
MANAGER_ID 0

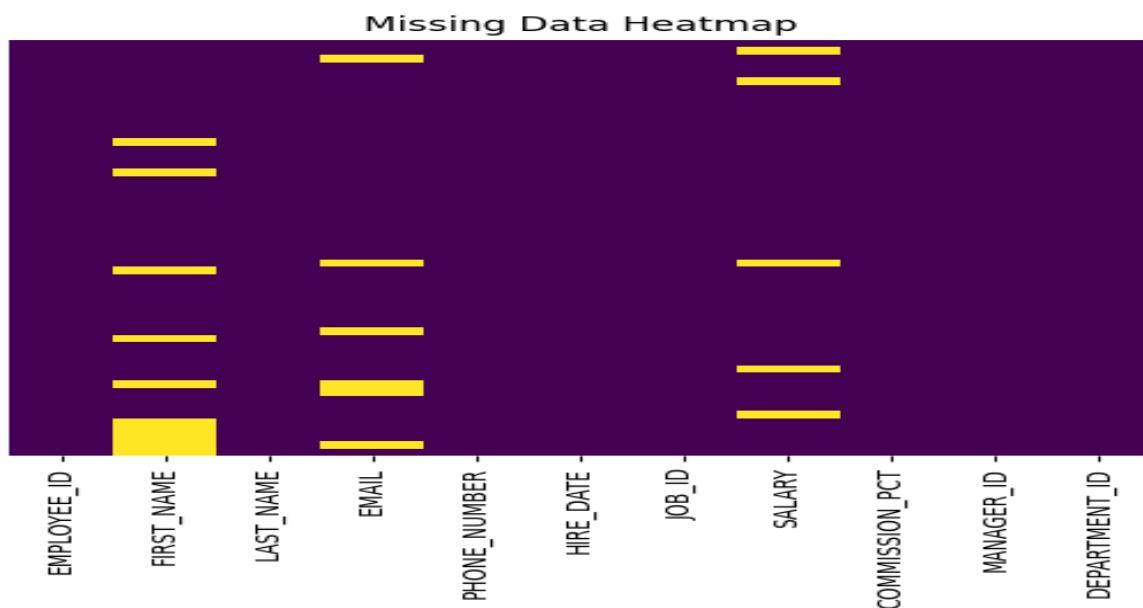
```

```

DEPARTMENT_ID 0
dtype: int64

Missing %:
EMPLOYEE_ID 0.000000
FIRST_NAME 18.181818
LAST_NAME 0.000000
EMAIL 10.909091
PHONE_NUMBER 0.000000
HIRE_DATE 0.000000
JOB_ID 0.000000
SALARY 9.090909
COMMISSION_PCT 0.000000
MANAGER_ID 0.000000
DEPARTMENT_ID 0.000000
dtype: float64

```



■ Program Code:

```

#op1
# Drop rows where SALARY is missing
df = df.dropna(subset=['SALARY'])

```

■ Output:

(No text output)

■ Program Code:

```

df['COMMISSION_PCT'] = df['COMMISSION_PCT'].replace(' - ', np.nan).infer_objects()
df['COMMISSION_PCT'] = pd.to_numeric(df['COMMISSION_PCT'], errors='coerce')

# Fill missing values
df['SALARY'] = df['SALARY'].fillna(df['SALARY'].mean())
df['COMMISSION_PCT'] = df['COMMISSION_PCT'].fillna(df['COMMISSION_PCT'].median())

# Optional
df['MANAGER_ID'] = df['MANAGER_ID'].fillna(df['MANAGER_ID'].mode()[0])
df['DEPARTMENT_ID'] = df['DEPARTMENT_ID'].fillna(df['DEPARTMENT_ID'].mode()[0])

```

■ Output:

(No text output)

■ Program Code:

```
#op3
# Fill categorical columns with mode
df['FIRST_NAME'] = df['FIRST_NAME'].fillna(df['FIRST_NAME'].mode()[0])
df['LAST_NAME'] = df['LAST_NAME'].fillna(df['LAST_NAME'].mode()[0])
df['EMAIL'] = df['EMAIL'].fillna(df['EMAIL'].mode()[0])
df['PHONE_NUMBER'] = df['PHONE_NUMBER'].fillna(df['PHONE_NUMBER'].mode()[0])
df['JOB_ID'] = df['JOB_ID'].fillna(df['JOB_ID'].mode()[0])
```

■ Output:

(No text output)

■ Program Code:

```
#step 4: method 1
Q1 = df['SALARY'].quantile(0.25)
Q3 = df['SALARY'].quantile(0.75)
IQR = Q3 - Q1

# Remove outliers
df_cleaned = df[~((df['SALARY'] < (Q1 - 1.5 * IQR)) |
                 (df['SALARY'] > (Q3 + 1.5 * IQR)))]
```

■ Output:

(No text output)

■ Program Code:

```
#method 2 z-scores
z_scores = np.abs(stats.zscore(df['SALARY']))

# Keep normal data only
df_no_outliers = df[(z_scores < 3)]
```

■ Output:

(No text output)

■ Program Code:

```
#Step 5: Handle Duplicates
# Count duplicates
print("Duplicate Rows:", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()
```

■ Output:

Duplicate Rows: 0

■ Program Code:

```
#Step 6: Correct Data Inconsistencies
df['FIRST_NAME'] = df['FIRST_NAME'].str.lower().str.strip()
df['LAST_NAME'] = df['LAST_NAME'].str.lower().str.strip()
```

```

df['EMAIL'] = df['EMAIL'].str.lower().str.strip()
df['JOB_ID'] = df['JOB_ID'].str.lower().str.strip()

df['JOB_ID'] = df['JOB_ID'].replace({
    'it_prog': 'it_programmer',
    'hr_rep': 'hr_representative'
})

# Remove invalid phone numbers
df = df[df['PHONE_NUMBER'] != 'INVALID']

# Fix negative salary
df['SALARY'] = df['SALARY'].abs()

```

■ Output:

(No text output)

■ Program Code:

```

print(df.head())
df.to_csv("emp1.csv")
print("file save")

```

■ Output:

```

EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER HIRE_DATE \
0 198 donald oconnell doconnel 650.507.9833 21-JUN-07
2 200 jennifer whalen afripp 515.123.4444 17-SEP-03
3 201 michael hartstein mhartste 515.123.5555 17-FEB-04
4 202 pat fay pfay 603.123.6666 17-AUG-05
6 204 hermann baer hbaer 515.123.8888 07-JUN-02

JOB_ID SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
0 sh_clerk 2600.0 NaN 124 50
2 ad_asst 4400.0 NaN 101 10
3 mk_man 13000.0 NaN 100 20
4 mk_rep 6000.0 NaN 201 20
6 pr_rep 10000.0 NaN 101 70
file save

```

Practical 12 — Data Characterization – Heart & Student Performance Dataset

■ Program Code:

```
import pandas as pd
df = pd.read_csv("characterization_dataset.csv")

print("First 5 Rows:\n", df.head())
print("\nInfo:\n")
print(df.info())

print("\nSummary:\n")
print(df.describe(include='all'))
```

■ Output:

```
First 5 Rows:
   EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER HIRE_DATE \
0 1 Priya NaN NaN INVALID 2020-01-01
1 2 NaN Kumar a@gmail.com NaN 2020-01-02
2 3 Neha Shah b@gmail.com 67890 2020-01-03
3 4 NaN NaN a@gmail.com 12345 2020-01-04
4 5 NaN NaN a@gmail.com INVALID 2020-01-05

   JOB_ID SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
0 IT_PROG NaN - 103.0 30.0
1 HR_REP 40000.0 NaN NaN 10.0
2 SALES -10000.0 0.2 101.0 10.0
3 HR_REP 40000.0 0.1 102.0 20.0
4 HR_REP 50000.0 0.2 102.0 30.0

Info:

<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 11 columns):
 # Column Non-Null Count Dtype
---  ---
0 EMPLOYEE_ID 100 non-null int64
1 FIRST_NAME 81 non-null str
2 LAST_NAME 76 non-null str
3 EMAIL 64 non-null str
4 PHONE_NUMBER 70 non-null str
5 HIRE_DATE 100 non-null str
6 JOB_ID 73 non-null str
7 SALARY 76 non-null float64
8 COMMISSION_PCT 75 non-null str
9 MANAGER_ID 77 non-null float64
10 DEPARTMENT_ID 75 non-null float64
dtypes: float64(3), int64(1), str(7)
memory usage: 12.2 KB
None

Summary:

   EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER \
```

```

count 100.000000 81 76 64 70
unique NaN 4 3 2 3
top NaN Priya Kumar a@gmail.com 12345
freq NaN 26 31 34 29
mean 50.500000 NaN NaN NaN NaN
std 29.011492 NaN NaN NaN NaN
min 1.000000 NaN NaN NaN NaN
25% 25.750000 NaN NaN NaN NaN
50% 50.500000 NaN NaN NaN NaN
75% 75.250000 NaN NaN NaN NaN
max 100.000000 NaN NaN NaN NaN

      HIRE_DATE JOB_ID SALARY COMMISSION_PCT MANAGER_ID \
count 100 73 76.000000 75 77.000000
unique 100 3 NaN 3 NaN
top 2020-01-01 HR_REP NaN 0.1 NaN
freq 1 31 NaN 34 NaN
mean NaN NaN 27763.157895 NaN 101.857143
std NaN NaN 22126.827410 NaN 0.822613
min NaN NaN -10000.000000 NaN 101.000000
25% NaN NaN 30000.000000 NaN 101.000000
50% NaN NaN 40000.000000 NaN 102.000000
75% NaN NaN 40000.000000 NaN 103.000000
max NaN NaN 50000.000000 NaN 103.000000

      DEPARTMENT_ID
count 75.000000
unique NaN
top NaN
freq NaN
mean 18.666667
std 8.274568
min 10.000000
25% 10.000000
50% 20.000000
75% 30.000000
max 30.000000

```

■ Program Code:

```

# step 2 : Check missing values count
print("\nMissing Values Count:\n", df.isnull().sum())

missing_percentage = df.isnull().mean() * 100
print("\nMissing %:\n", missing_percentage)

```

■ Output:

```

Missing Values Count:
  EMPLOYEE_ID 0
  FIRST_NAME 19
  LAST_NAME 24
  EMAIL 36
  PHONE_NUMBER 30
  HIRE_DATE 0
  JOB_ID 27

```

```

SALARY 24
COMMISSION_PCT 25
MANAGER_ID 23
DEPARTMENT_ID 25
dtype: int64

Missing %:
EMPLOYEE_ID 0.0
FIRST_NAME 19.0
LAST_NAME 24.0
EMAIL 36.0
PHONE_NUMBER 30.0
HIRE_DATE 0.0
JOB_ID 27.0
SALARY 24.0
COMMISSION_PCT 25.0
MANAGER_ID 23.0
DEPARTMENT_ID 25.0
dtype: float64

```

■ Program Code:

```

#Step 1: Inspect Data Types & Summary
# Column types
print(df.dtypes)
# Descriptive statistics
print(df.describe(include='all'))

```

■ Output:

```

EMPLOYEE_ID int64
FIRST_NAME str
LAST_NAME str
EMAIL str
PHONE_NUMBER str
HIRE_DATE str
JOB_ID str
SALARY float64
COMMISSION_PCT str
MANAGER_ID float64
DEPARTMENT_ID float64
dtype: object

      EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER \
count 100.000000  81  76  64  70
unique NaN  4  3  2  3
top NaN Priya Kumar a@gmail.com 12345
freq NaN 26 31 34 29
mean 50.500000 NaN NaN NaN NaN
std 29.011492 NaN NaN NaN NaN
min 1.000000 NaN NaN NaN NaN
25% 25.750000 NaN NaN NaN NaN
50% 50.500000 NaN NaN NaN NaN
75% 75.250000 NaN NaN NaN NaN
max 100.000000 NaN NaN NaN NaN

      HIRE_DATE JOB_ID SALARY COMMISSION_PCT MANAGER_ID \
count 100  73  76.000000  75  77.000000

```

```

unique 100 3 NaN 3 NaN
top 2020-01-01 HR_REP NaN 0.1 NaN
freq 1 31 NaN 34 NaN
mean NaN NaN 27763.157895 NaN 101.857143
std NaN NaN 22126.827410 NaN 0.822613
min NaN NaN -10000.000000 NaN 101.000000
25% NaN NaN 30000.000000 NaN 101.000000
50% NaN NaN 40000.000000 NaN 102.000000
75% NaN NaN 40000.000000 NaN 103.000000
max NaN NaN 50000.000000 NaN 103.000000

```

```

DEPARTMENT_ID
count 75.000000
unique NaN
top NaN
freq NaN
mean 18.666667
std 8.274568
min 10.000000
25% 10.000000
50% 20.000000
75% 30.000000
max 30.000000

```

■ Program Code:

```

#Step 3: Explore Data from Multiple Sources
# Example: Combine multiple CSV files
df1 = pd.read_csv('Heart.csv')
df2 = pd.read_csv('StudentsPerformance.csv')
# Combine vertically
df_combined = pd.concat([df1, df2], ignore_index=True)
# Combine horizontally (on key)
df_merged = pd.merge(df1, df2, on='gender', how='inner')
print("Combine vertically : \n",df_combined)
print("\nCombine horizontally : \n",df_merged)

```

■ Output:

```

Combine vertically :
   Unnamed: 0  Age  Sex  ChestPain  RestBP  Chol  Fbs  RestECG  MaxHR \
0  1.0  63.0  1.0  typical  145.0  233.0  1.0  2.0  150.0
1  2.0  67.0  1.0  asymptomatic  160.0  286.0  0.0  2.0  108.0
2  3.0  67.0  1.0  asymptomatic  120.0  229.0  0.0  2.0  129.0
3  4.0  37.0  1.0  nonanginal  130.0  250.0  0.0  0.0  187.0
4  5.0  41.0  0.0  nontypical  130.0  204.0  0.0  2.0  172.0
..  ..  ..  ..  ..  ..  ..  ..  ..
194 NaN NaN NaN NaN NaN NaN NaN NaN NaN
195 NaN NaN NaN NaN NaN NaN NaN NaN NaN
196 NaN NaN NaN NaN NaN NaN NaN NaN NaN
197 NaN NaN NaN NaN NaN NaN NaN NaN NaN
198 NaN NaN NaN NaN NaN NaN NaN NaN NaN

   ExAng ...  Thal  AHD  gender  race/ethnicity \
0  0.0  ...  fixed  No  female  NaN
1  1.0  ...  normal  Yes  female  NaN
2  1.0  ...  reversable  Yes  female  NaN

```

```

3 0.0 ... normal No male NaN
4 0.0 ... normal No male NaN
... ..
194 NaN ... NaN NaN female group B
195 NaN ... NaN NaN male group C
196 NaN ... NaN NaN male group B
197 NaN ... NaN NaN female group E
198 NaN ... NaN NaN female group D

    parental level of education lunch test preparation course \
0 NaN NaN NaN
1 NaN NaN NaN
2 NaN NaN NaN
3 NaN NaN NaN
4 NaN NaN NaN
... ..
194 some college standard none
195 associate's degree free/reduced completed
196 some high school standard completed
197 some college standard completed
198 some college free/reduced none

    math score reading score writing score
0 NaN NaN NaN
1 NaN NaN NaN
2 NaN NaN NaN
3 NaN NaN NaN
4 NaN NaN NaN
... ..
194 79.0 86.0 92.0
195 78.0 81.0 82.0
196 65.0 66.0 62.0
197 63.0 72.0 70.0
198 58.0 67.0 62.0

[199 rows x 23 columns]

Combine horizontally :
    Unnamed: 0 Age Sex ChestPain RestBP Chol Fbs RestECG MaxHR \
0 1 63 1 typical 145 233 1 2 150
1 1 63 1 typical 145 233 1 2 150
2 1 63 1 typical 145 233 1 2 150
3 1 63 1 typical 145 233 1 2 150
4 1 63 1 typical 145 233 1 2 150
... ..
4896 99 52 1 nontypical 134 201 0 0 158
4897 99 52 1 nontypical 134 201 0 0 158
4898 99 52 1 nontypical 134 201 0 0 158
4899 99 52 1 nontypical 134 201 0 0 158
4900 99 52 1 nontypical 134 201 0 0 158

    ExAng ... Thal AHD gender race/ethnicity \
0 0 ... fixed No female group B
1 0 ... fixed No female group C
2 0 ... fixed No female group B

```

```

3 0 ... fixed No female group B
4 0 ... fixed No female group B
... ..
4896 0 ... normal No female group D
4897 0 ... normal No female group C
4898 0 ... normal No female group B
4899 0 ... normal No female group E
4900 0 ... normal No female group D

    parental level of education lunch test preparation course \
0 bachelor's degree standard none
1 some college standard completed
2 master's degree standard none
3 associate's degree standard none
4 some college standard completed
... ..
4896 some high school standard none
4897 bachelor's degree standard none
4898 some college standard none
4899 some college standard completed
4900 some college free/reduced none

    math score reading score writing score
0 72 72 74
1 69 90 88
2 90 95 93
3 71 83 78
4 88 95 92
... ..
4896 73 86 82
4897 65 72 74
4898 79 86 92
4899 63 72 70
4900 58 67 62

[4901 rows x 23 columns]

```

■ Program Code:

```

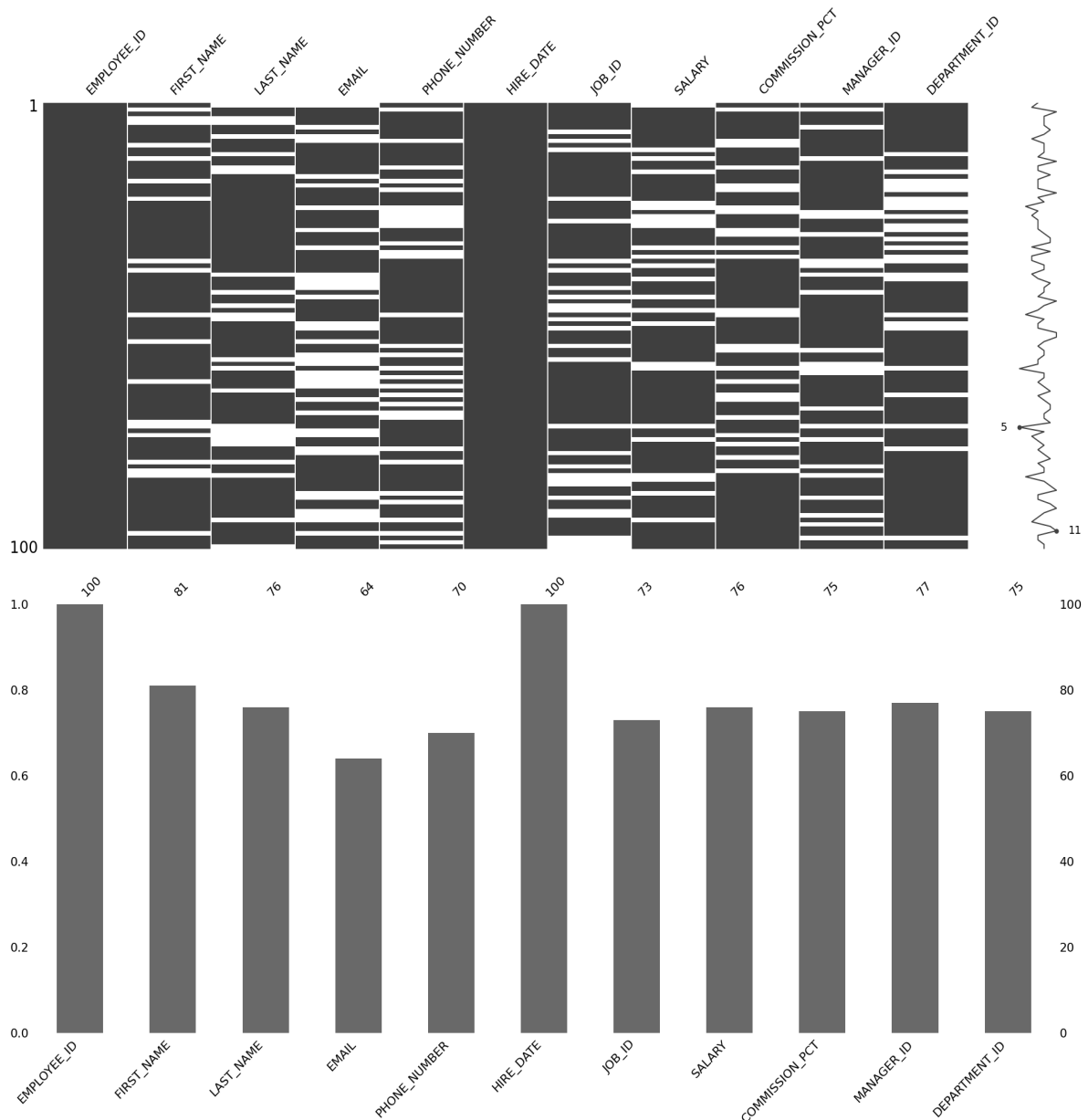
#Step 4: Visualize Missing Patterns
import matplotlib.pyplot as plt
import missingno as msno
import seaborn as sns

# Matrix plot
msno.matrix(df)
plt.show()

# Bar chart
msno.bar(df)
plt.show()

```

■ Output:



■ Program Code:

```
#Step 5: Summary Statistics for Characterization
# Categorical column distribution
for col in df.select_dtypes(include='object'):
    print(df[col].value_counts(normalize=True) * 100)
# Correlation matrix for numerical columns
print(df.corr(numeric_only=True))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```

■ Output:

```
FIRST_NAME
Priya 32.098765
Ravi 25.925926
Amit 22.222222
Neha 19.753086
Name: proportion, dtype: float64
LAST_NAME
Kumar 40.789474
```

```

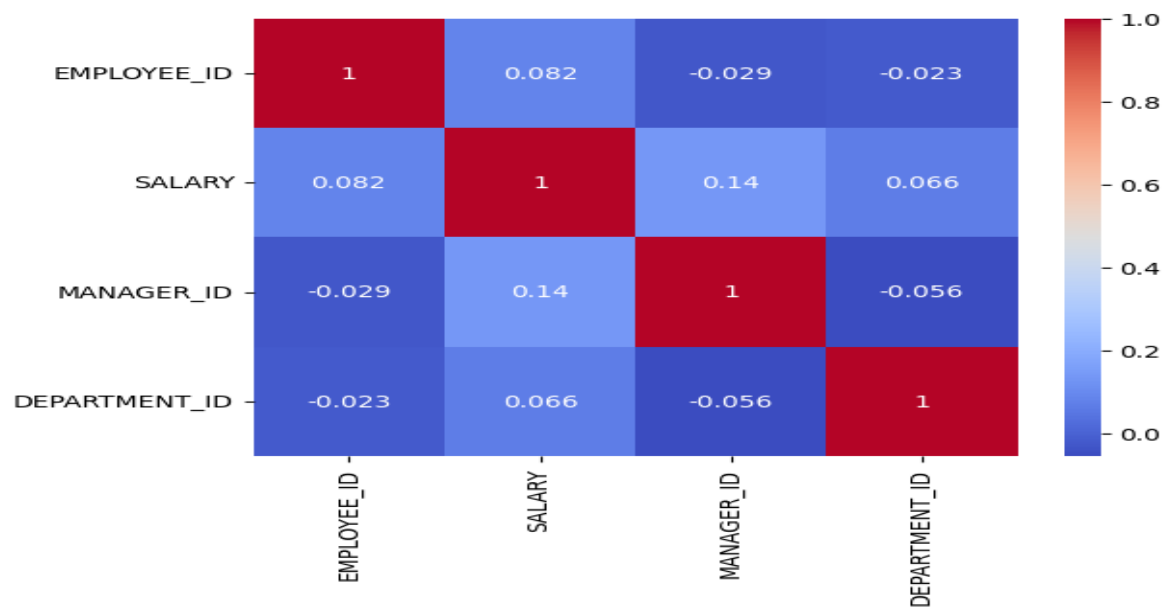
Shah 32.894737
Patel 26.315789
Name: proportion, dtype: float64
EMAIL
a@gmail.com 53.125
b@gmail.com 46.875
Name: proportion, dtype: float64
PHONE_NUMBER
12345 41.428571
67890 30.000000
INVALID 28.571429
Name: proportion, dtype: float64
HIRE_DATE
2020-01-01 1.0
2020-01-02 1.0
2020-01-03 1.0
2020-01-04 1.0
2020-01-05 1.0
...
2020-04-05 1.0
2020-04-06 1.0
2020-04-07 1.0
2020-04-08 1.0
2020-04-09 1.0
Name: proportion, Length: 100, dtype: float64
JOB_ID
HR_REP 42.465753
SALES 32.876712
IT_PROG 24.657534
Name: proportion, dtype: float64
COMMISSION_PCT
0.1 45.333333
- 34.666667
0.2 20.000000
Name: proportion, dtype: float64
      EMPLOYEE_ID SALARY MANAGER_ID DEPARTMENT_ID
EMPLOYEE_ID 1.000000 0.082436 -0.028947 -0.022531
SALARY 0.082436 1.000000 0.137643 0.065614
MANAGER_ID -0.028947 0.137643 1.000000 -0.055680
DEPARTMENT_ID -0.022531 0.065614 -0.055680 1.000000

```

/tmp/ipykernel_1269/3910812795.py:3: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.

See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
for col in df.select_dtypes(include='object'):
```



Practical 13 — Data Transformation & Standardization – Employee Dataset

■ Program Code:

```
# Program 13: Data Transformation & Standardization
# Step 1: Load Dataset
import pandas as pd
import numpy as np

# Step 1: Load Dataset
df = pd.read_csv('employee.csv')

print("First 5 Rows:")
print(df.head())

print("\nDataset Info:")
print(df.info())
```

■ Output:

```
First 5 Rows:
   Company Age Salary Place Country Gender Department
0  Infosys 30.0   NaN  Mumbai   India    0      HR
1   TCS     35.0 2300.0  Calcutta  India    0      IT
2  Infosys 40.0 3000.0   Delhi   India    0  Finance
3   TCS     23.0 4000.0  Mumbai   India    0      IT
4  Infosys  NaN 5000.0  Calcutta  India    0      HR

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 32 entries, 0 to 31
Data columns (total 7 columns):
 # Column Non-Null Count Dtype
---  ---
0 Company 32 non-null str
1 Age 28 non-null float64
2 Salary 27 non-null float64
3 Place 32 non-null str
4 Country 32 non-null str
5 Gender 32 non-null int64
6 Department 32 non-null str
dtypes: float64(2), int64(1), str(4)
memory usage: 2.5 KB
None
```

■ Program Code:

```
# Step 2: Handle Missing Values

# Convert blank values to NaN
df.replace("", np.nan, inplace=True)

# Fill missing Age with mean
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Fill missing Salary with median
```

```
df['Salary'] = df['Salary'].fillna(df['Salary'].median())

# Fill missing Place with mode
df['Place'] = df['Place'].fillna(df['Place'].mode()[0])
```

■ Output:

(No text output)

■ Program Code:

```
# Step 3: Encode Categorical Data

# Label Encoding for Company, Place, Country
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Department'] = le.fit_transform(df['Department'])

# Gender is already numeric (0/1), so no need

print("\nAfter Encoding:")
print(df.head())

#One-Hot Encoding
df = pd.get_dummies(df, columns=['Gender'], drop_first=True)
```

■ Output:

```
After Encoding:
   Company Age Salary Place Country Gender Department
0  Infosys 30.00 4824.0  Mumbai  India  0    1
1   TCS    35.00 2300.0  Calcutta  India  0    2
2  Infosys 40.00 3000.0   Delhi   India  0    0
3   TCS    23.00 4000.0  Mumbai   India  0    2
4  Infosys 32.25 5000.0  Calcutta  India  0    1
```

■ Program Code:

```
# Step 4: Scale or Normalize Numerical Columns
# Standard Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])
```

■ Output:

(No text output)

■ Program Code:

```
#Min-Max Scaling
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])
```

■ Output:

(No text output)

■ Program Code:

```
# Step 5: Subset Features for Analysis
# Select only relevant columns

df_subset = df[['Age', 'Salary', 'Department', 'Gender_1']]
print(df_subset.head())
```

■ Output:

```
      Age Salary Department Gender_1
0  0.375000  0.425060  1 False
1  0.531250  0.137817  2 False
2  0.687500  0.217480  0 False
3  0.156250  0.331285  2 False
4  0.445312  0.445089  1 False
```

■ Program Code:

```
print("\nFinal Cleaned Dataset:")
print(df.head())
```

■ Output:

```
Final Cleaned Dataset:
      Company Age Salary Place Country Department Gender_1
0  Infosys  0.375000  0.425060  Mumbai  India  1 False
1   TCS  0.531250  0.137817  Calcutta  India  2 False
2  Infosys  0.687500  0.217480   Delhi  India  0 False
3   TCS  0.156250  0.331285  Mumbai  India  2 False
4  Infosys  0.445312  0.445089  Calcutta  India  1 False
```

■ Program Code:

```
# Step 6: Split Dataset into Analytical Subsets

dept_groups = df.groupby('Department')
for dept, group in dept_groups:
    print(f"\nDepartment: {dept}")
    print(group.head())
```

■ Output:

```
Department: 0
      Company Age Salary Place Country Department Gender_1
2  Infosys  0.687500  0.217480   Delhi  India  0 False
6  Infosys  0.156250  0.672698  Mumbai  India  0 True
8   CTS  0.843750  0.900307   Delhi  India  0 False
11  CTS  0.843750  0.425060  Chennai  India  0 False
14  TCS  0.156250  0.217480   Delhi  India  0 False

Department: 1
      Company Age Salary Place Country Department Gender_1
0  Infosys  0.375000  0.425060  Mumbai  India  1 False
4  Infosys  0.445312  0.445089  Calcutta  India  1 False
9   CTS  0.156250  0.425060  Mumbai  India  1 False
12  CTS  0.000000  0.016502  Mumbai  India  1 False
15  Infosys  0.156250  0.220895  Podicherry India  1 False

Department: 2
```

```

Company Age Salary Place Country Department Gender_1
1 TCS 0.531250 0.137817 Calcutta India 2 False
3 TCS 0.156250 0.331285 Mumbai India 2 False
5 TCS 0.445312 0.558894 Chennai India 2 True
7 TCS 0.500000 0.786503 Calcutta India 2 True
10 CTS 0.500000 0.000000 Calcutta India 2 False

```

■ Program Code:

```

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,4))

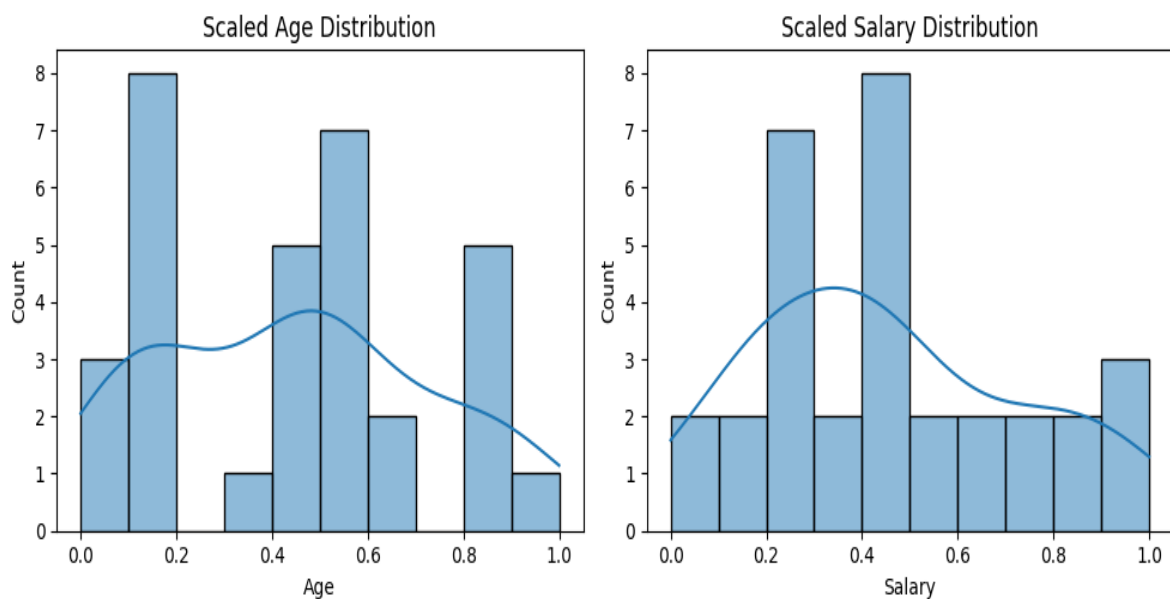
plt.subplot(1,2,1)
sns.histplot(df['Age'], kde=True, bins=10)
plt.title('Scaled Age Distribution')

plt.subplot(1,2,2)
sns.histplot(df['Salary'], kde=True, bins=10)
plt.title('Scaled Salary Distribution')

plt.tight_layout()
plt.show()

```

■ Output:



■ Program Code:

```

print(df.head())
df.to_csv("emp1.csv")
print("file save")

```

■ Output:

```

Company Age Salary Place Country Department Gender_1
0 Infosys 0.375000 0.425060 Mumbai India 1 False
1 TCS 0.531250 0.137817 Calcutta India 2 False
2 Infosys 0.687500 0.217480 Delhi India 0 False

```

```
3 TCS 0.156250 0.331285 Mumbai India 2 False
4 Infosys 0.445312 0.445089 Calcutta India 1 False
file save
```

Practical 14 — Data Loading and Exploration – Student Dataset

■ Program Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the dataset (Use your lab's .csv file here)
#df = pd.read_csv("student_data.csv")
# For demonstration, we'll use a sample dataset
df = sns.load_dataset("iris")

# 2. Initial Data Inspection
print("--- First 5 Rows ---")
print(df.head()) # Shows the top rows
print("\n--- Data Structure & Missing Values ---")
print(df.info()) # Checks data types and null values

# 3. Descriptive Statistics
print("\n--- Summary Statistics ---")
print(df.describe()) # Shows mean, median, std dev

# 4. Univariate Analysis (Distribution of a single variable)
plt.figure(figsize=(6, 4))
sns.distplot(df['sepal_length'], kde=True)
plt.title('Distribution of Sepal Length')
plt.show()

# 5. Bivariate Analysis (Relationship between two variables)
# Scatter Plot (Matplotlib)
plt.figure(figsize=(6, 4))

for species in df['species'].unique():
    subset = df[df['species'] == species]
    plt.scatter(subset['sepal_length'], subset['sepal_width'], label=species)

plt.legend()
plt.title("Sepal Length vs Width")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()

plt.figure(figsize=(8, 5))

corr = df.corr() # FIXED LINE

sns.heatmap(corr, annot=True)
plt.title('Correlation Heatmap')

plt.show()
```

■ Output:

```
HTTPError: HTTP Error 403: Forbidden
```

Practical 15 — Structured Data Summarization – Scottish Hills Dataset

■ Program Code:

```
import pandas as pd
import matplotlib.pyplot as plt# 1. Load Tabular Data
# Use pd.read_csv("your_file.csv") for external files
df = pd.read_csv("scottish_hills.csv")
# 2. Structured Summarization
print("--- Dataset Snapshot ---")
print(df.head(10)) # First 10 rows for initial interpretation
print("\n--- Summary Statistics ---")
# Generates mean, median, min, max, and std deviation
print(df.describe())
# 3. Categorical Interpretation
# Example: Grouping by a category to see trends (e.g., Average height per Region)
if 'Region' in df.columns:
    print("\n--- Region-wise Summary ---")
    print(df.groupby('Region')['Height'].mean())
# 4. Visual Interpretation (Box Plot)
# Useful for seeing distributions and identifying outliers
df.boxplot(column='Height')
plt.title('Distribution of Heights')
plt.ylabel('Meters')
plt.show()
```

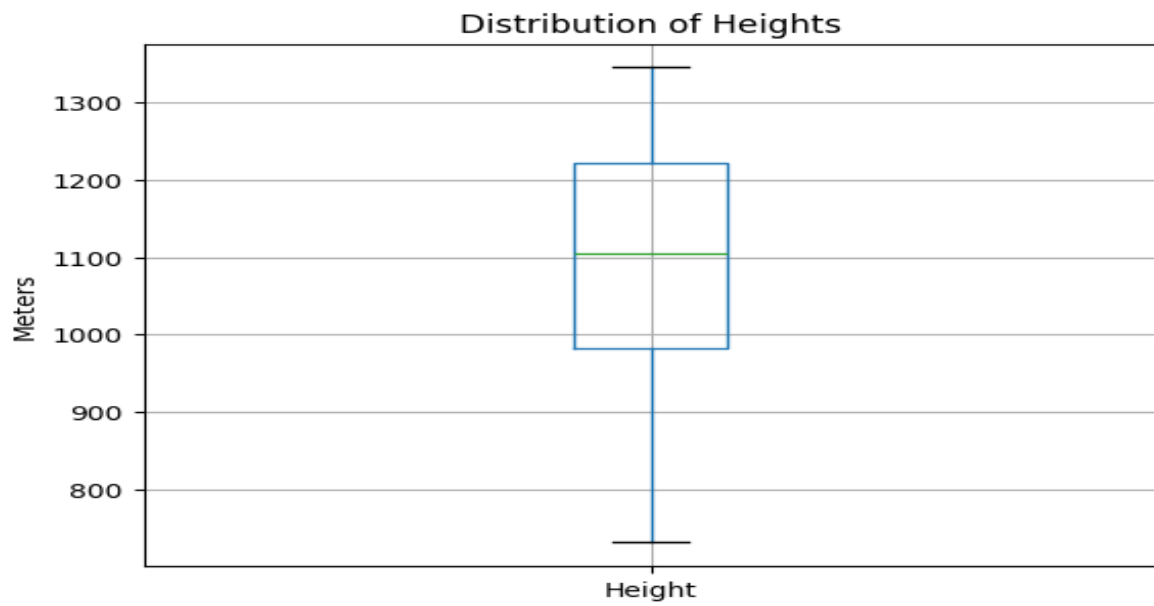
■ Output:

```
--- Dataset Snapshot ---
   Region Hill Latitude Longitude Height
0 Highlands Ben Nevis 56.7969 -5.0036 1345
1 Cairngorms Braeriach 57.0783 -3.7283 1296
2 Cairngorms Sgòr an Lochain Uaine 57.0570 -3.7260 1258
3 Cairngorms Cairn Toul 57.0542 -3.7100 1291
4 Cairngorms Cairn Gorm 57.1167 -3.6444 1245
5 Highlands Aonach Beag 56.8333 -4.5290 1234
6 Highlands Aonach Mòr 56.8167 -4.9610 1221
7 Highlands Carn Mòr Dearg 56.8052 -4.9866 1220
8 Perthshire Ben Lawers 56.5447 -4.2200 1214
9 Cairngorms Ben Macdui 57.0704 -3.6683 1309

--- Summary Statistics ---
           Latitude Longitude Height
count 30.000000 30.000000 30.000000
mean 57.021333 -4.777920 1098.133333
std 0.569712 0.779353 148.468209
min 55.626400 -6.223800 731.000000
25% 56.669775 -5.238275 982.250000
50% 57.033850 -4.973800 1104.500000
75% 57.211425 -4.129100 1220.750000
max 58.394200 -3.619100 1345.000000

--- Region-wise Summary ---
Region
Cairngorms 1244.428571
Highlands 1072.000000
```

```
Isle of Arran 874.000000  
Isle of Mull 966.000000  
Isle of Skye 985.000000  
Loch Lomond 974.000000  
Perthshire 1148.500000  
Name: Height, dtype: float64
```



Practical 16 — JSON Data Loading and Structured Output – CSV Conversion

■ Program Code:

```
import pandas as pd
import json

# STEP 1: Load JSON data from file
with open('data.json', 'r') as f:
    data = json.load(f)

# STEP 2: Manually Flatten JSON (NO json_normalize)
flat_data = []

for item in data:
    new_row = {
        "id": item["id"],
        "name": item["info"]["name"],
        "price": item["info"]["price"],
        "tags": item["tags"] # tags is now a single string, avoiding duplicates
    }
    flat_data.append(new_row)

df_final = pd.DataFrame(flat_data)

print("=== Step 3: Flattened Data ===")
print(df_final)

# STEP 4: Summarization
print("\n=== Statistics (Price) ===")
print(df_final["price"].describe())

# STEP 5: Export CSV
df_final.to_csv("structured_output.csv", index=False)

print("\nFile 'structured_output.csv' created successfully!")
```

■ Output:

```
=== Step 3: Flattened Data ===
   id name price tags
0 1 Laptop 1200 electronics, office
1 2 Chair 300 furniture, home
2 3 Monitor 500 electronics
3 4 Desk 700 furniture, office
4 5 Mouse 50 electronics, accessories
5 6 Keyboard 100 electronics, accessories
6 7 Office Lamp 40 furniture, office
7 8 Headphones 150 electronics, accessories
8 9 Bookshelf 200 furniture, home
9 10 Web Camera 80 electronics, accessories
10 11 Running Shoes 120 clothing, sports
11 12 Apple 2 food, groceries
12 13 Novel 15 books, entertainment
13 14 Tennis Racket 85 sports, equipment
14 15 T-Shirt 20 clothing, casual
```

```
=== Statistics (Price) ===  
count 15.000000  
mean 237.466667  
std 330.518827  
min 2.000000  
25% 45.000000  
50% 100.000000  
75% 250.000000  
max 1200.000000  
Name: price, dtype: float64  
  
File 'structured_output.csv' created successfully!
```

Practical 17 — Data Cleaning – Raw Employee Data Preprocessing

■ Program Code:

```
import pandas as pd
import numpy as np

# STEP 1: Load Data
df = pd.read_csv("raw_data.csv")

print("\n--- Step 1: Raw Data ---")
print(df.head(15))

# STEP 2: Remove Duplicates (based on Employee_ID)
df = df.drop_duplicates(subset="Employee_ID")

# STEP 3: Handle Missing Values
df['Name'] = df['Name'].fillna('Unknown')

# Age cleaning
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

# Replace invalid ages (<18 or >60) with NaN
df.loc[(df['Age'] < 18) | (df['Age'] > 60), 'Age'] = np.nan

# Fill with mean
df['Age'] = df['Age'].fillna(df['Age'].mean())

# STEP 4: Salary Cleaning
df['Salary'] = pd.to_numeric(df['Salary'], errors='coerce')

# Remove negative salary
df.loc[df['Salary'] < 0, 'Salary'] = np.nan

# Remove extreme outliers (optional threshold)
df.loc[df['Salary'] > 200000, 'Salary'] = np.nan

# Fill with median
df['Salary'] = df['Salary'].fillna(df['Salary'].median())

# STEP 5: Text Standardization
df['Name'] = df['Name'].str.title()

# STEP 6: Date Cleaning (Important Fix)
df['Join_Date'] = pd.to_datetime(df['Join_Date'], errors='coerce', dayfirst=True)

# Optional: fill missing dates with most frequent date
df['Join_Date'] = df['Join_Date'].fillna(df['Join_Date'].mode()[0])

# STEP 7: Reset Index
df = df.reset_index(drop=True)

print("\n--- Final Cleaned Data ---")
print(df.head(15))
```

```
# Save cleaned data
df.to_csv("cleaned_employee_data.csv", index=False)

print("\n Cleaned dataset saved successfully!")
```

■ Output:

```
--- Step 1: Raw Data ---
   Employee_ID Name Age Salary Join_Date
0  101  anil kumar  28.0  50000  2023-01-10
1  102  Sunita  32.0  60000  2023/05/12
2  103   NaN  150.0  75000  12-08-2023
3  101  anil kumar  28.0  50000  2023-01-10
4  105  Rahul  NaN missing  2023-09-01
5  106  amit  45.0  45000  15/07/2023
6  107  Neha  29.0  52000  2023-02-14
7  108  Vikram  40.0  78000  2023/06/18
8  109  Suresh  35.0  -1000  05-09-2023
9  110  pooja  24.0  48000  2022/12/01
10 111  Karan  31.0  65000  10/10/2023
11 112   NaN  27.0 missing  2023-11-20
12 113  Sneha  200.0  85000  21/04/2023
13 114  Ramesh  38.0  62000  2023-03-30
14 115  Ramesh  38.0  62000  2023-03-30

--- Final Cleaned Data ---
   Employee_ID Name Age Salary Join_Date
0  101  Anil Kumar  28.000000  50000.0  2023-10-01
1  102  Sunita  32.000000  60000.0  2023-01-09
2  103  Unknown  33.805556  75000.0  2023-01-09
3  105  Rahul  33.805556  55000.0  2023-01-09
4  106  Amit  45.000000  45000.0  2023-01-09
5  107  Neha  29.000000  52000.0  2023-01-09
6  108  Vikram  40.000000  78000.0  2023-01-09
7  109  Suresh  35.000000  55000.0  2023-01-09
8  110  Pooja  24.000000  48000.0  2023-01-09
9  111  Karan  31.000000  65000.0  2023-01-09
10 112  Unknown  27.000000  55000.0  2023-01-09
11 113  Sneha  33.805556  85000.0  2023-01-09
12 114  Ramesh  38.000000  62000.0  2023-01-09
13 115  Ramesh  38.000000  62000.0  2023-01-09
14 116  Gita  25.000000  55000.0  2023-01-09

Cleaned dataset saved successfully!
```

Practical 18 — Statistical Data Analysis – Dataset Creation & Summary

■ Program Code:

```
import pandas as pd
import numpy as np

# STEP 1: Create Dataset

df = pd.read_csv("data.csv")

print("--- Step 1: Basic Dataset ---")
print(df.head())

# STEP 2: Enhancement (Feature Engineering)
df['Profit'] = df['Revenue'] - df['Operating_Costs']
df['Margin_%'] = (df['Profit'] / df['Revenue'] * 100)

print("\n--- Step 2: Enhanced Data ---")
print(df[['Branch', 'Revenue', 'Profit', 'Margin_%']])

# STEP 3: Scaling (Min-Max Normalization)
min_rev = df['Revenue'].min()
max_rev = df['Revenue'].max()

df['Rev_Score'] = (df['Revenue'] - min_rev) / (max_rev - min_rev)

print("\n--- Step 3: Normalized Revenue ---")
print(df[['Branch', 'Revenue', 'Rev_Score']])

# STEP 4: Pivot Table (Restructuring)
pivot_df = df.pivot_table(index='Branch', columns='Month', values='Profit', aggfunc='mean')

print("\n--- Step 4: Pivot Table (Profit by Branch & Month) ---")
print(pivot_df)

# STEP 5: Aggregation Summary
summary = df.groupby('Branch').agg({
    'Revenue': 'sum',
    'Profit': 'sum',
    'Margin_%': 'mean'
})

summary = summary.rename(columns={'Margin_%': 'Avg_Margin'})

print("\n--- Step 5: Final Summary ---")
print(summary)
```

■ Output:

```
--- Step 1: Basic Dataset ---
  Branch Month Revenue Operating_Costs
0 East  Jan  5000    3000
1 West  Jan  4500    2800
2 North Jan  7100    4000
```

```
3 South Jan 4800 2500
4 East Feb 6200 3200

--- Step 2: Enhanced Data ---
    Branch Revenue Profit Margin_%
0 East 5000 2000 40.000000
1 West 4500 1700 37.777778
2 North 7100 3100 43.661972
3 South 4800 2300 47.916667
4 East 6200 3000 48.387097
5 West 3900 800 20.512821
6 North 6800 3000 44.117647
7 South 5300 2600 49.056604
8 East 5100 2000 39.215686
9 West 4600 1700 36.956522
10 North 7200 3100 43.055556
11 South 4900 2300 46.938776
12 East 6300 3000 47.619048
13 West 4000 800 20.000000
14 North 6900 3000 43.478261
15 South 5400 2600 48.148148
16 East 5200 2000 38.461538
17 West 4700 1700 36.170213
18 North 7300 3100 42.465753
19 South 5000 2300 46.000000
20 East 6400 3000 46.875000
21 West 4100 800 19.512195
22 North 7000 3000 42.857143
23 South 5500 2600 47.272727
24 East 5300 2000 37.735849
25 West 4800 1700 35.416667
26 North 7400 3100 41.891892
27 South 5100 2300 45.098039
28 East 6500 3000 46.153846
29 West 4200 800 19.047619
30 North 7100 3000 42.253521
31 South 5600 2600 46.428571
32 East 5400 2000 37.037037
33 West 4900 1700 34.693878
34 North 7500 3100 41.333333
35 South 5200 2300 44.230769
36 East 6600 3000 45.454545
37 West 4300 800 18.604651
38 North 7200 3000 41.666667
39 South 5700 2600 45.614035
40 East 5500 2000 36.363636
41 West 5000 1700 34.000000
42 North 7600 3100 40.789474
43 South 5300 2300 43.396226
44 East 6700 3000 44.776119
45 West 4400 800 18.181818
46 North 7300 3000 41.095890
47 South 5800 2600 44.827586
48 East 5200 2100 40.384615
49 West 4550 1650 36.263736
```

```
--- Step 3: Normalized Revenue ---
```

```
    Branch Revenue Rev_Score
```

```
0 East 5000 0.297297
1 West 4500 0.162162
2 North 7100 0.864865
3 South 4800 0.243243
4 East 6200 0.621622
5 West 3900 0.000000
6 North 6800 0.783784
7 South 5300 0.378378
8 East 5100 0.324324
9 West 4600 0.189189
10 North 7200 0.891892
11 South 4900 0.270270
12 East 6300 0.648649
13 West 4000 0.027027
14 North 6900 0.810811
15 South 5400 0.405405
16 East 5200 0.351351
17 West 4700 0.216216
18 North 7300 0.918919
19 South 5000 0.297297
20 East 6400 0.675676
21 West 4100 0.054054
22 North 7000 0.837838
23 South 5500 0.432432
24 East 5300 0.378378
25 West 4800 0.243243
26 North 7400 0.945946
27 South 5100 0.324324
28 East 6500 0.702703
29 West 4200 0.081081
30 North 7100 0.864865
31 South 5600 0.459459
32 East 5400 0.405405
33 West 4900 0.270270
34 North 7500 0.972973
35 South 5200 0.351351
36 East 6600 0.729730
37 West 4300 0.108108
38 North 7200 0.891892
39 South 5700 0.486486
40 East 5500 0.432432
41 West 5000 0.297297
42 North 7600 1.000000
43 South 5300 0.378378
44 East 6700 0.756757
45 West 4400 0.135135
46 North 7300 0.918919
47 South 5800 0.513514
48 East 5200 0.351351
49 West 4550 0.175676
```

```
--- Step 4: Pivot Table (Profit by Branch & Month) ---
```

```
Month Apr Aug Dec Feb Jan Jul Jun Mar \
Branch
East 3000.0 3000.0 3000.0 3000.0 2050.0 2000.0 3000.0 2000.0
North 3000.0 3000.0 3000.0 3000.0 3100.0 3100.0 3000.0 3100.0
South 2600.0 2600.0 2600.0 2600.0 2300.0 2300.0 2600.0 2300.0
West 800.0 800.0 800.0 800.0 1675.0 1700.0 800.0 1700.0
```

```
Month May Nov Oct Sep
Branch
East 2000.0 2000.0 3000.0 2000.0
North 3100.0 3100.0 3000.0 3100.0
South 2300.0 2300.0 2600.0 2300.0
West 1700.0 1700.0 800.0 1700.0
```

```
--- Step 5: Final Summary ---
      Revenue Profit Avg_Margin
Branch
East 75400 32100 42.189540
North 86400 36600 42.388926
South 63600 29400 46.244012
West 57950 16650 28.241377
```

Practical 19 — Seaborn Visualization – Multi-Plot Data Analysis

■ Program Code:

```
#seaborn and sns uske karine karvano baki che e karine lakhvo

import pandas as pd
import matplotlib.pyplot as plt

# STEP 1: Dataset

df = pd.read_csv("data.csv")

print("--- Dataset ---")
print(df)

# STEP 2: LINE PLOT (Trend)
plt.figure()

for product in df['Product'].unique():
    subset = df[df['Product'] == product]
    plt.plot(subset['Month'], subset['Sales'], marker='o', label=product)

plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.legend()
plt.show()

# STEP 3: BAR CHART (Comparison)
plt.figure()

grouped = df.groupby('Product')['Sales'].sum()
plt.bar(grouped.index, grouped.values)

plt.title('Total Sales Comparison')
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.show()

# STEP 4: SCATTER PLOT (Relationship)
plt.figure()

for product in df['Product'].unique():
    subset = df[df['Product'] == product]
    plt.scatter(subset['Advertising_Spend'], subset['Sales'], label=product)

plt.title('Advertising vs Sales')
plt.xlabel('Advertising Spend')
plt.ylabel('Sales')
plt.legend()
plt.show()

# STEP 5: BOX PLOT (Distribution)
```

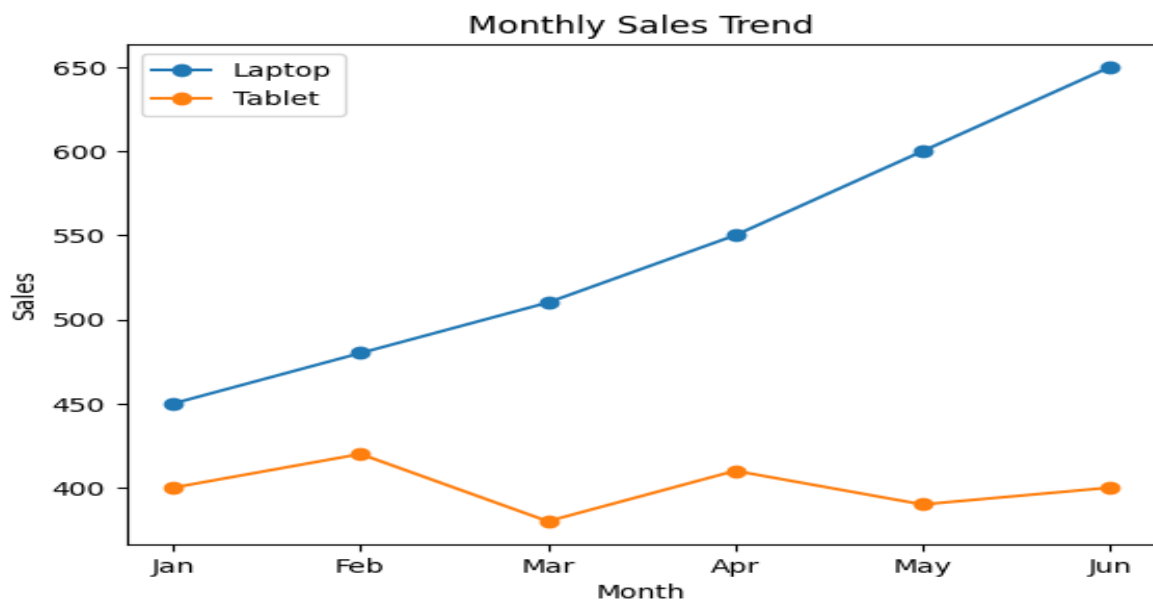
```
plt.figure()

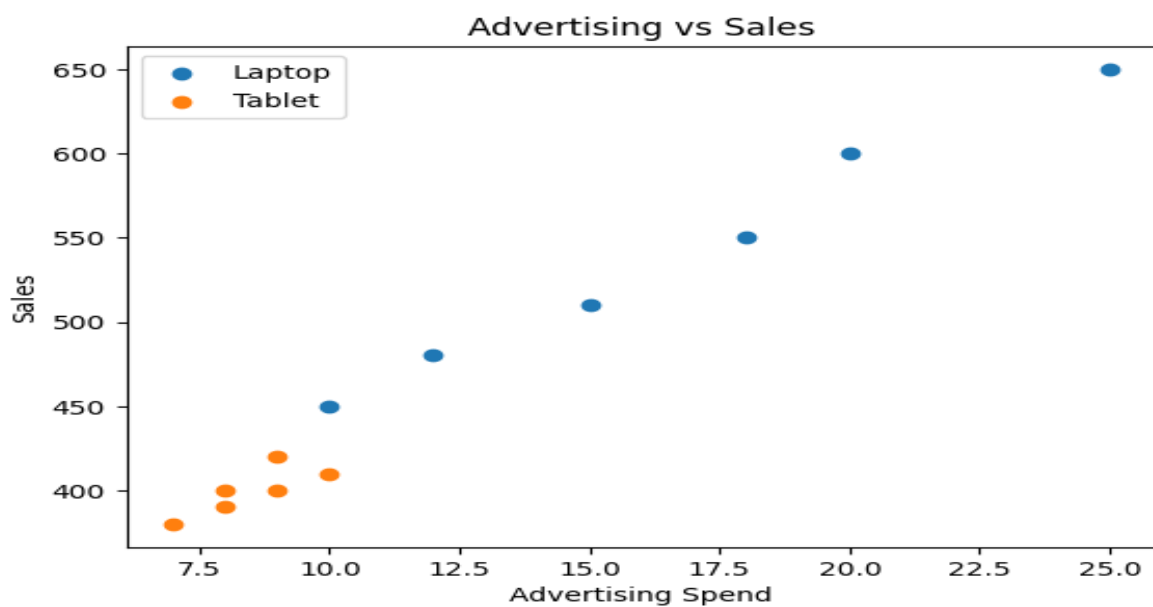
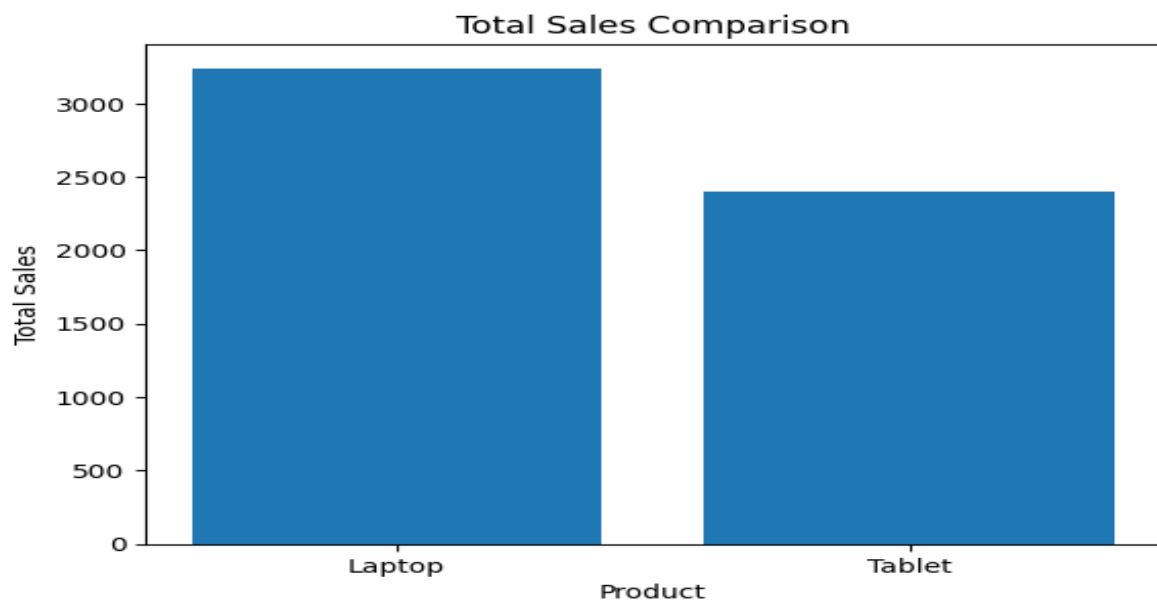
df.boxplot(column='Sales', by='Product')

plt.title('Sales Distribution')
plt.suptitle("") # remove extra title
plt.xlabel('Product')
plt.ylabel('Sales')
plt.show()
```

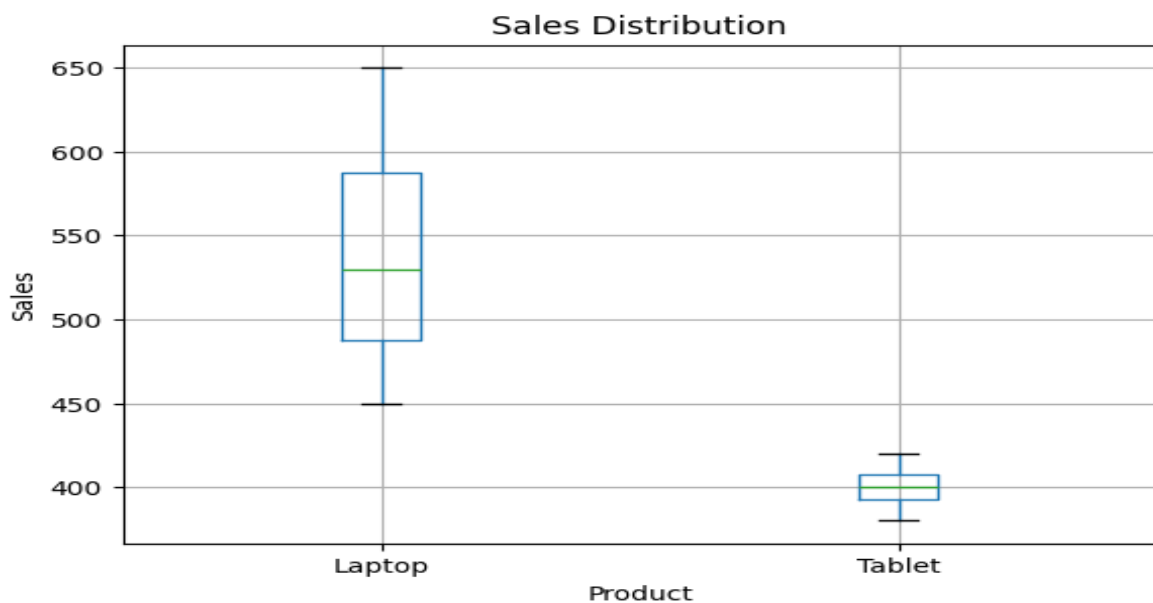
■ Output:

```
--- Dataset ---
  Month Sales Product Advertising_Spend
0 Jan  450 Laptop  10
1 Feb  480 Laptop  12
2 Mar  510 Laptop  15
3 Apr  550 Laptop  18
4 May  600 Laptop  20
5 Jun  650 Laptop  25
6 Jan  400 Tablet  8
7 Feb  420 Tablet  9
8 Mar  380 Tablet  7
9 Apr  410 Tablet 10
10 May  390 Tablet 8
11 Jun  400 Tablet 9
```





<Figure size 640x480 with 0 Axes>



Practical 20 — Score Dataset Analysis – Statistical Operations

■ Program Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# STEP 1: Create Dataset

df = pd.read_csv("scores.csv")

print("--- Step 1: Raw Scores ---")
print(df.T)

# STEP 2: Central Tendency
mean_val = df['Scores'].mean()
median_val = df['Scores'].median()
mode_val = df['Scores'].mode()[0]

print("\n--- Step 2: Central Tendency ---")
print("Mean:", mean_val)
print("Median:", median_val)
print("Mode:", mode_val)

# STEP 3: Dispersion
range_val = df['Scores'].max() - df['Scores'].min()
variance_val = df['Scores'].var()
std_dev = df['Scores'].std()

print("\n--- Step 3: Dispersion ---")
print("Range:", range_val)
print("Variance:", variance_val)
print("Standard Deviation:", round(std_dev, 2))

# STEP 4: Quartiles
quartiles = df['Scores'].quantile([0.25, 0.5, 0.75])

print("\n--- Step 4: Quartiles ---")
print(quartiles)

# STEP 5: Box Plot (Matplotlib - Safe)
plt.figure()

plt.boxplot(df['Scores'], vert=False)

plt.title('Statistical Spread of Exam Scores')
plt.xlabel('Scores')

plt.show()
```

■ Output:

```
--- Step 1: Raw Scores ---
  0  1  2  3  4  5  6  7  8  9  ...  40  41  42  43  44  45 \
```

```

Scores 85 88 90 78 92 45 89 95 82 80 ... 84 87 91 92 78 80

      46 47 48 49
Scores 83 85 88 100

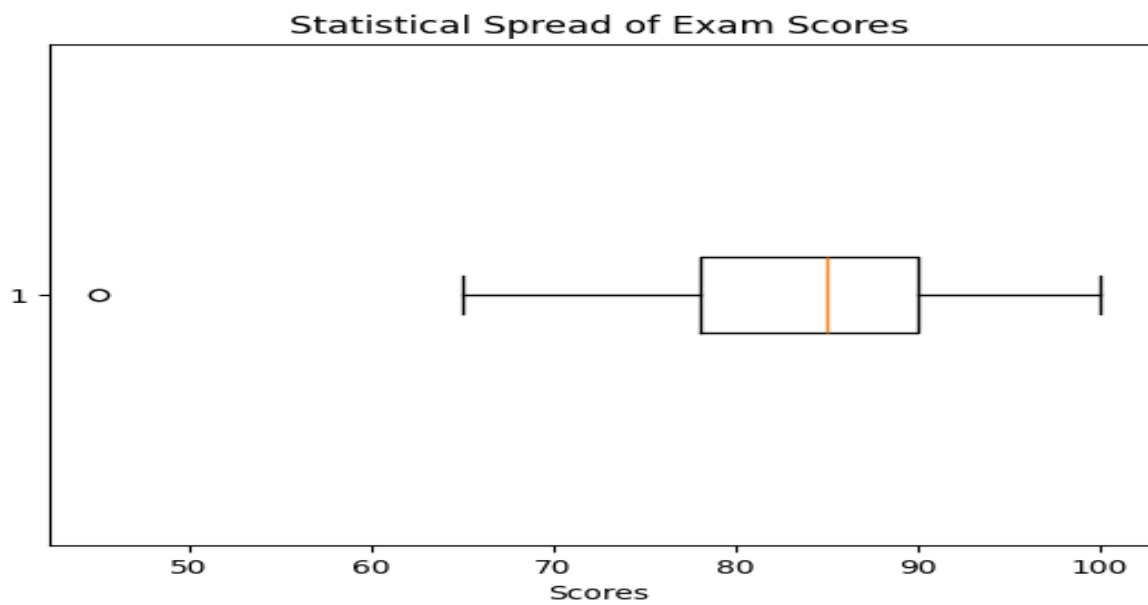
[1 rows x 50 columns]

--- Step 2: Central Tendency ---
Mean: 82.94
Median: 85.0
Mode: 88

--- Step 3: Dispersion ---
Range: 55
Variance: 97.69020408163267
Standard Deviation: 9.88

--- Step 4: Quartiles ---
0.25 78.0
0.50 85.0
0.75 90.0
Name: Scores, dtype: float64

```



Practical 21 — Data Analysis with Visualization – Lab Data

■ Program Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# --- STEP 1: DATA PREPARATION ---

df = pd.read_csv("data.csv")

print("--- STEP 1: DATASET ---")
print(df.head(15))

# --- STEP 2: CORRELATION ---
corr_matrix = df.corr()

print("\n--- STEP 2: CORRELATION COEFFICIENTS ---")
print(corr_matrix['Final_Grade'].round(4))

# --- STEP 3: REGRESSION MODEL ---
X = df[['Study_Hrs']]
Y = df['Final_Grade']

model = LinearRegression()
model.fit(X, Y)

m = model.coef_[0]
c = model.intercept_
r_sq = model.score(X, Y)

print("\n--- STEP 3: REGRESSION PARAMETERS ---")
print("Slope (m):", m)
print("Intercept (c):", c)
print("R-Squared:", round(r_sq, 4))

# --- STEP 4: PREDICTION (FIXED) ---
new_input = np.array([[11]]) #
prediction = model.predict(new_input)

print("\n--- STEP 4: PREDICTION ---")
print("Predicted Grade for 11 Study Hours:", round(prediction[0], 2))

# --- STEP 5: VISUALIZATION (MATPLOTLIB ONLY) ---

# 1. Scatter Plot
plt.figure()
plt.scatter(df['Study_Hrs'], df['Final_Grade'])

# Regression Line
y_pred_line = model.predict(X)
plt.plot(df['Study_Hrs'], y_pred_line)
```

```
plt.title("Study Hours vs Final Grade")
plt.xlabel("Study Hours")
plt.ylabel("Final Grade")
plt.show()

# 2. Correlation Heatmap (manual)
plt.figure()
plt.imshow(corr_matrix, cmap='coolwarm')

plt.colorbar()
plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns)
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

plt.title("Correlation Matrix")
plt.show()
```

■ Output:

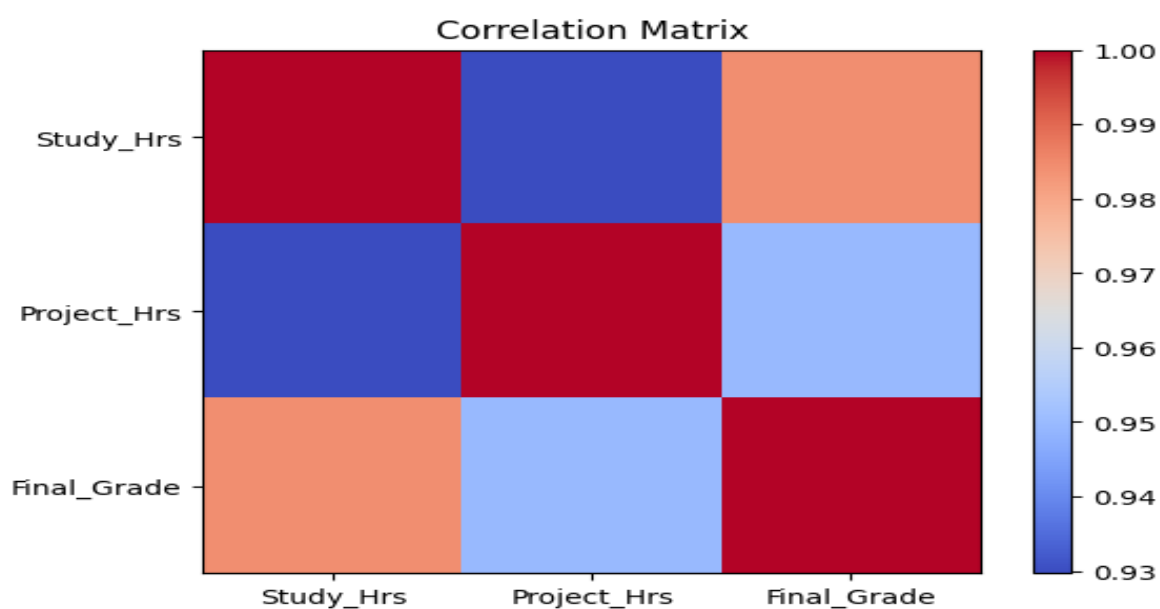
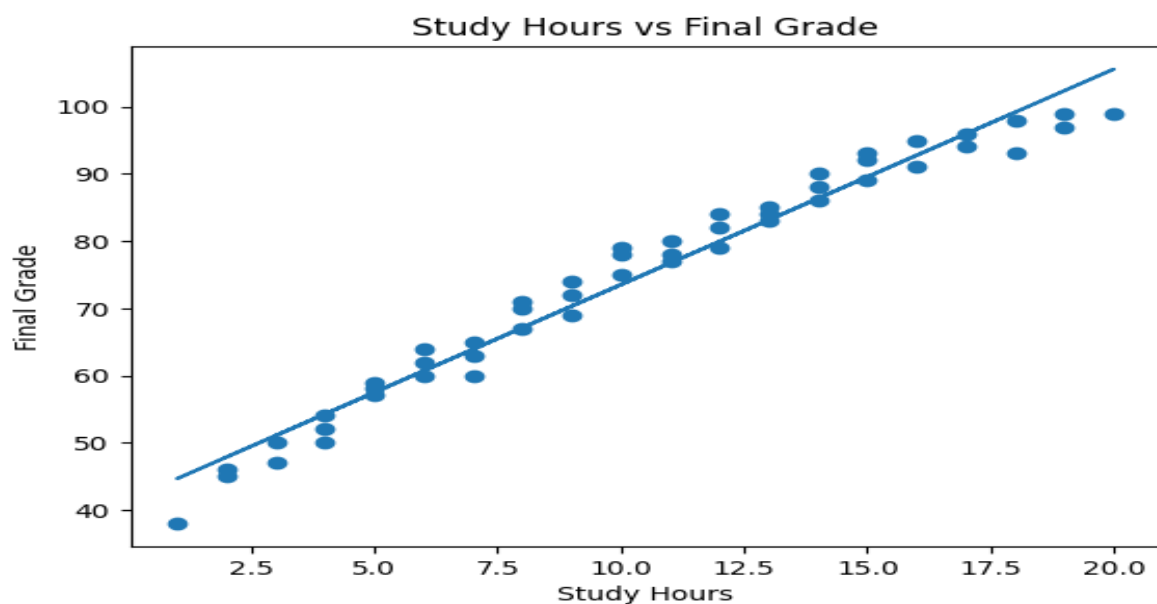
```
--- STEP 1: DATASET ---
  Study_Hrs Project_Hrs Final_Grade
0 2 1 45
1 4 2 52
2 5 3 58
3 7 5 65
4 8 4 70
5 10 6 78
6 12 8 82
7 14 7 88
8 15 9 92
9 18 10 98
10 3 1 47
11 4 3 54
12 5 2 57
13 6 4 62
14 6 2 60

--- STEP 2: CORRELATION COEFFICIENTS ---
Study_Hrs 0.9842
Project_Hrs 0.9496
Final_Grade 1.0000
Name: Final_Grade, dtype: float64

--- STEP 3: REGRESSION PARAMETERS ---
Slope (m): 3.207218696701033
Intercept (c): 41.43393617184741
R-Squared: 0.9687

--- STEP 4: PREDICTION ---
Predicted Grade for 11 Study Hours: 76.71

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2691: UserWarning: X does not have
valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```



Practical 22 — Time Series Analysis – ADF Test & Stationarity

■ Program Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller

# STEP 1: Create Time Dataset

np.random.seed(42)

# 7 days data (30 min interval)
time_index = pd.date_range(
    start='2024-05-01',
    periods=336,
    freq='30min'
)

# Trend component
trend = np.linspace(22, 28, 336)

# Seasonal component
seasonal = 3 * np.sin(2 * np.pi * time_index.hour / 24)

# Noise component
noise = np.random.normal(0, 0.4, 336)

# Final sensor values
sensor_values = trend + seasonal + noise

# Create DataFrame
df = pd.DataFrame(
    {'Temp_Celsius': sensor_values},
    index=time_index
)

print(df.head())

# STEP 2: Stationarity Test

print("\n--- ADF TEST ---")

result = adfuller(df['Temp_Celsius'])

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] <= 0.05:
```

```
        print("Stationary Data")
    else:
        print("Non-Stationary Data")

# STEP 3: Time Series Decomposition

decomposition = seasonal_decompose(
    df['Temp_Celsius'],
    model='additive',
    period=48
)

# Create Component Table
df_components = pd.DataFrame({
    'Observed': decomposition.observed,
    'Trend': decomposition.trend,
    'Seasonal': decomposition.seasonal,
    'Residual': decomposition.resid
})

print("\nFirst 5 Rows:")
print(df_components.head())

# STEP 4: Visualization

fig, axes = plt.subplots(4, 1, figsize=(12,10), sharex=True)

# Observed
df_components['Observed'].plot(
    ax=axes[0],
    color='blue',
    title='Observed Data'
)

# Trend
df_components['Trend'].plot(
    ax=axes[1],
    color='red',
    title='Trend'
)

# Seasonal
df_components['Seasonal'].plot(
    ax=axes[2],
    color='green',
    title='Seasonality'
)

# Residual
df_components['Residual'].plot(
    ax=axes[3],
    color='black',
    style='.',
```

```

        title='Residuals'
    )

plt.tight_layout()
plt.show()

```

■ Output:

```

Temp_Celsius
2024-05-01 00:00:00 22.198686
2024-05-01 00:30:00 21.962605
2024-05-01 01:00:00 23.071353
2024-05-01 01:30:00 23.439400
2024-05-01 02:00:00 23.477980

```

--- ADF TEST ---

ADF Statistic: -1.6196251282105936

p-value: 0.4729258343759594

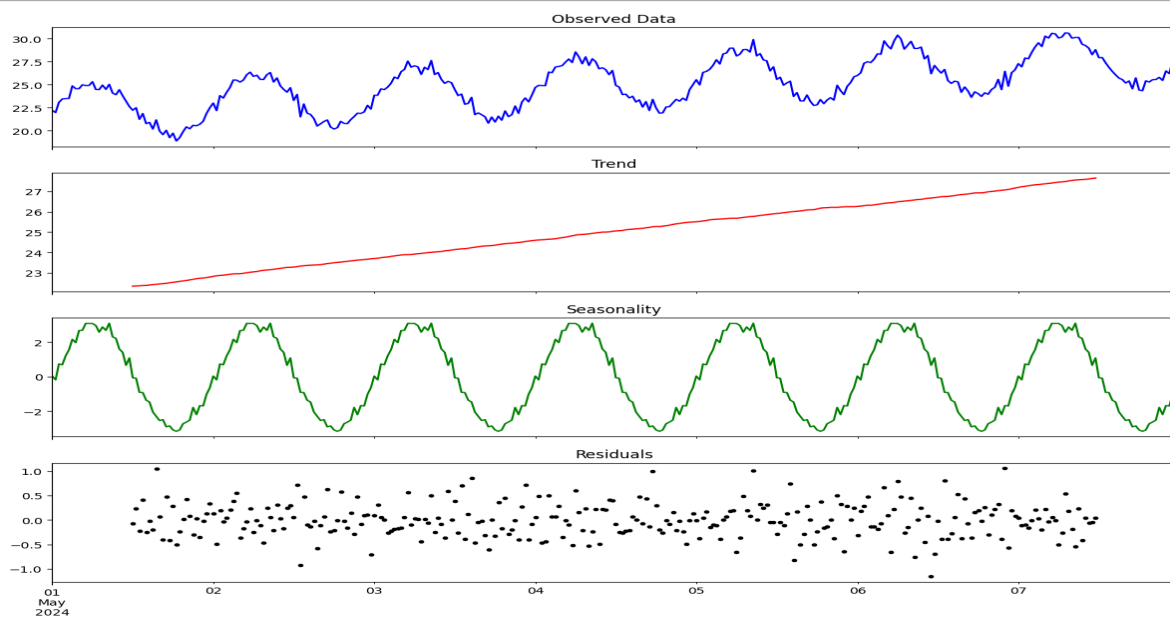
Non-Stationary Data

First 5 Rows:

```

Observed Trend Seasonal Residual
2024-05-01 00:00:00 22.198686 NaN 0.033915 NaN
2024-05-01 00:30:00 21.962605 NaN -0.187892 NaN
2024-05-01 01:00:00 23.071353 NaN 0.737413 NaN
2024-05-01 01:30:00 23.439400 NaN 0.686330 NaN
2024-05-01 02:00:00 23.477980 NaN 1.205108 NaN

```



Practical 23 — Time Series Exploration – Sensor Data

■ Program Code:

```
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller

# STEP 1: Load Dataset

df = pd.read_csv('lab_sensor.csv', parse_dates=['timestamp'], index_col='timestamp')

print(df.head())

# STEP 2: Stationarity Test

adf_res = adfuller(df['reading'])

print("ADF p-value:", adf_res[1])

if adf_res[1] > 0.05:
    print("Trend or seasonality exists")
else:
    print("Data is stationary")

# STEP 3: Time Series Decomposition

decomp = seasonal_decompose(
    df['reading'],
    model='additive',
    period=24
)

# STEP 4: Visualization

decomp.plot()
plt.show()
```

■ Output:

```
/tmp/ipykernel_1315/3801497592.py:10: UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify
a format.
```

```
df = pd.read_csv('lab_sensor.csv', parse_dates=['timestamp'], index_col='timestamp')
```

	reading
timestamp	
2001-05-24 00:00:00	22.5
2001-05-24 01:00:00	23.1
2001-05-24 02:00:00	22.8

```
2001-05-24 03:00:00 24.0  
2001-05-24 04:00:00 24.3  
ADF p-value: 1.7032467395735897e-05  
Data is stationary
```



Practical 24 — Lab Data Preparation – Cleaning & Feature Engineering

■ Program Code:

```
import pandas as pd

# STEP 1: Load Dataset

df = pd.read_csv('lab_data.csv', parse_dates=['ts'])

print("Original Data:")
print(df.head())

# STEP 2: Set Timestamp Index

df = df.set_index('ts')

# Sort timestamps
df = df.sort_index()

print("\nIndexed Data:")
print(df.head())

# STEP 3: Resampling

# Convert data into 15-minute intervals
df_prepared = df.resample('15T').mean()

print("\nAfter Resampling:")
print(df_prepared.head(10))

# STEP 4: Interpolation

# Fill missing values
df_prepared['value'] = df_prepared['value'].interpolate(method='time')

print("\nAfter Interpolation:")
print(df_prepared.head(10))

# STEP 5: Save Prepared Data

df_prepared.to_csv('prepared_lab_data.csv')

print("\nPrepared dataset saved successfully.")
```

■ Output:

```
Original Data:
              ts  value
0 2001-05-24 00:00:00  22.5
1 2001-05-24 01:00:00  23.1
```

```
2 2001-05-24 02:00:00 22.8
3 2001-05-24 03:00:00 24.0
4 2001-05-24 04:00:00 24.3
```

Indexed Data:

	value
ts	
2001-05-24 00:00:00	22.5
2001-05-24 01:00:00	23.1
2001-05-24 02:00:00	22.8
2001-05-24 03:00:00	24.0
2001-05-24 04:00:00	24.3

```
/tmp/ipykernel_1335/2952745709.py:6: UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify
a format.
```

```
df = pd.read_csv('lab_data.csv', parse_dates=['ts'])
```

```
ValueError: Invalid frequency: 15T. Failed to parse with error message: ValueError("Invalid frequency: T.
Failed to parse with error message: KeyError('T'). Did you mean min?")
```

Practical 25 — Sensor Data Pipeline – Ingestion, Cache & Processing

■ Program Code:

```
import pandas as pd
from statsmodels.tsa.seasonal import seasonal_decompose

# MODULE 1: INGESTION

def ingestion_module(input_file, output_file):

    df = pd.read_csv(input_file)

    # Correct timestamp format
    df['ts'] = pd.to_datetime(
        df['ts'],
        format='%Y/%m/%d %H:%M'
    )

    # Set timestamp index
    df.set_index('ts', inplace=True)

    # Save parquet
    df.to_parquet(output_file)

    return "Ingestion Complete"

# MODULE 2: PREPARATION

def preparation_module(input_file):

    df = pd.read_parquet(input_file)

    # Sort timestamps
    if not df.index.is_monotonic_increasing:
        df = df.sort_index()

    # Resampling
    df_structured = df.resample('15min').mean()

    # Interpolation
    df_structured['reading'] = df_structured[
        'reading'
    ].interpolate(method='time')

    return df_structured

# MODULE 3: EXPLORATION
```

```
def exploration_module(df, period_val=96):

    analysis = seasonal_decompose(
        df['reading'],
        model='additive',
        period=period_val
    )

    stats = df['reading'].describe()

    return analysis, stats


# MAIN EXECUTION

print(
    ingestion_module(
        'lab_sensor.csv',
        'sensor_cache.parquet'
    )
)

prepared_df = preparation_module(
    'sensor_cache.parquet'
)

print(prepared_df.head())

analysis, stats = exploration_module(
    prepared_df
)

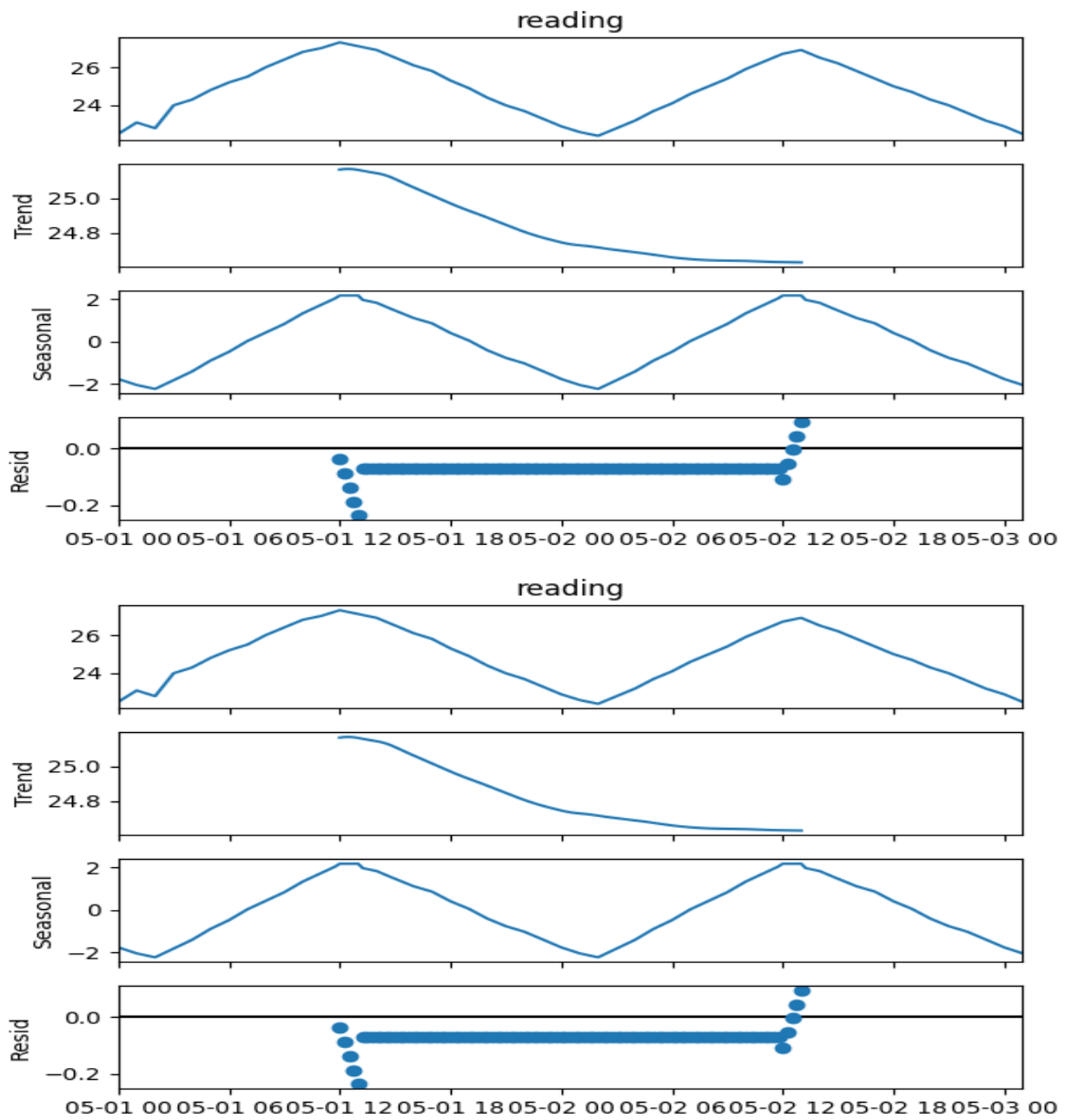
print(stats)

analysis.plot()
```

■ Output:

```
Ingestion Complete

              reading
ts
2024-05-01 00:00:00 22.50
2024-05-01 00:15:00 22.65
2024-05-01 00:30:00 22.80
2024-05-01 00:45:00 22.95
2024-05-01 01:00:00 23.10
count 197.000000
mean 24.841117
std 1.440370
min 22.400000
25% 23.600000
50% 24.900000
75% 26.100000
max 27.300000
Name: reading, dtype: float64
```



Practical 26 — Lab Data Analytics – Exploratory Analysis

■ Program Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score
)

# STEP 1: LOAD DATA

df = pd.read_csv('lab_sensor.csv')

# STEP 2: FEATURE ENGINEERING

# Create features

df['Delta'] = df['reading'].diff().fillna(0)

df['Rolling_Var'] = (
    df['reading']
    .rolling(window=5)
    .var()
    .fillna(0)
)

# STEP 3: CREATE CLASS LABELS

mean = df['reading'].mean()
std = df['reading'].std()

conditions = [
    (df['reading'] <= mean + std),

    (df['reading'] > mean + std) &
    (df['reading'] <= mean + 2 * std),

    (df['reading'] > mean + 2 * std)
]

df['Class_Label'] = np.select(
    conditions,
    [0, 1, 2],
    default=0
```

```
)

# STEP 4: FEATURE & TARGET

X = df[['reading', 'Delta', 'Rolling_Var']]

y = df['Class_Label']

# STEP 5: TRAIN TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# STEP 6: MODEL TRAINING

clf = RandomForestClassifier(
    n_estimators=150,
    max_depth=10,
    random_state=42
)

clf.fit(X_train, y_train)

# STEP 7: PREDICTION

y_pred = clf.predict(X_test)

# STEP 8: EVALUATION

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred
    )
)
```

■ Output:

Accuracy:

```
0.9
```

```
Confusion Matrix:
```

```
[[8 1]
 [0 1]]
```

```
Classification Report:
```

```
precision recall f1-score support
```

```
0 1.00 0.89 0.94 9
```

```
1 0.50 1.00 0.67 1
```

```
accuracy 0.90 10
```

```
macro avg 0.75 0.94 0.80 10
```

```
weighted avg 0.95 0.90 0.91 10
```

Practical 27 — Analytical Data Cache – Processing & Summary

■ Program Code:

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# STEP 1: LOAD DATA

df = pd.read_csv(
    'lab_analytical_cache.csv'
)

print(df.head())

# STEP 2: FEATURE SELECTION

X = df[['reading', 'Delta_Change']]

# STEP 3: FEATURE SCALING

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# STEP 4: ELBOW METHOD

inertia_scores = []

K_range = range(1, 11)

for k in K_range:

    model = KMeans(
        n_clusters=k,
        init='k-means++',
        n_init=10,
        random_state=42
    )

    model.fit(X_scaled)

    inertia_scores.append(model.inertia_)

# Plot Elbow Curve
```

```
plt.figure(figsize=(8,4))

plt.plot(
    K_range,
    inertia_scores,
    'bx-'
)

plt.xlabel('Number of Clusters')

plt.ylabel('Inertia')

plt.title('Elbow Method')

plt.show()

# STEP 5: FINAL CLUSTERING

optimal_k = 3

final_model = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    n_init=10,
    random_state=42
)

cluster_labels = final_model.fit_predict(
    X_scaled
)

# Add labels

df['Inherent_Cluster'] = cluster_labels

print("\nClustered Data:")
print(df.head())

# STEP 6: SILHOUETTE SCORE

score = silhouette_score(
    X_scaled,
    cluster_labels
)

print("\nSilhouette Score:")
print(score)

# STEP 7: VISUALIZATION

plt.figure(figsize=(8,5))
```

```
plt.scatter(
    df['reading'],
    df['Delta_Change'],
    c=df['Inherent_Cluster']
)

plt.xlabel('Reading')

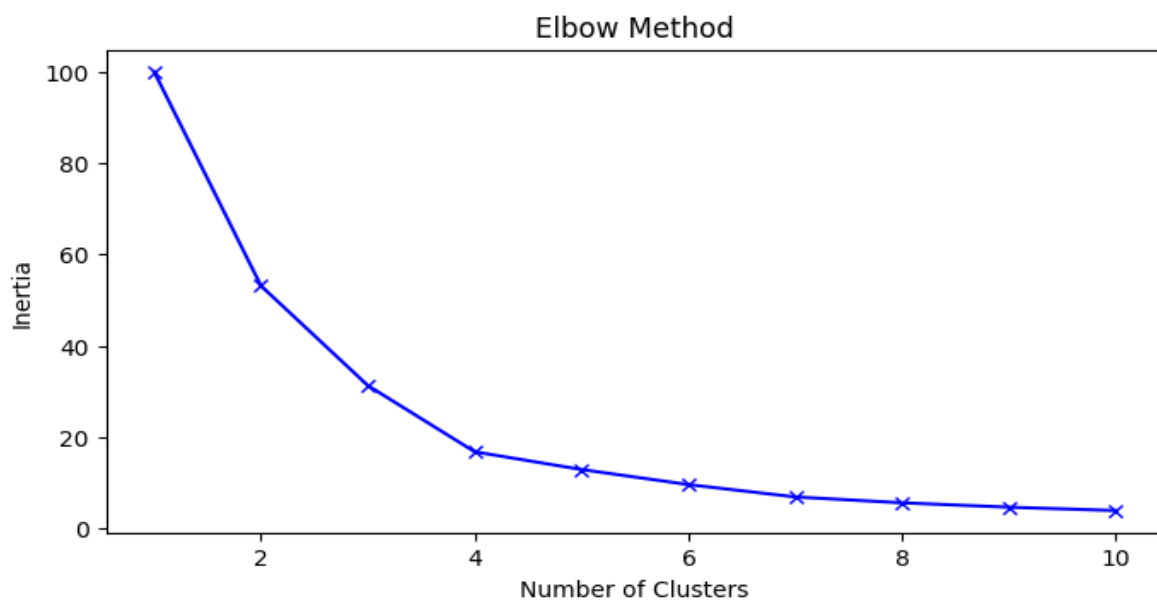
plt.ylabel('Delta Change')

plt.title('Cluster Visualization')

plt.show()
```

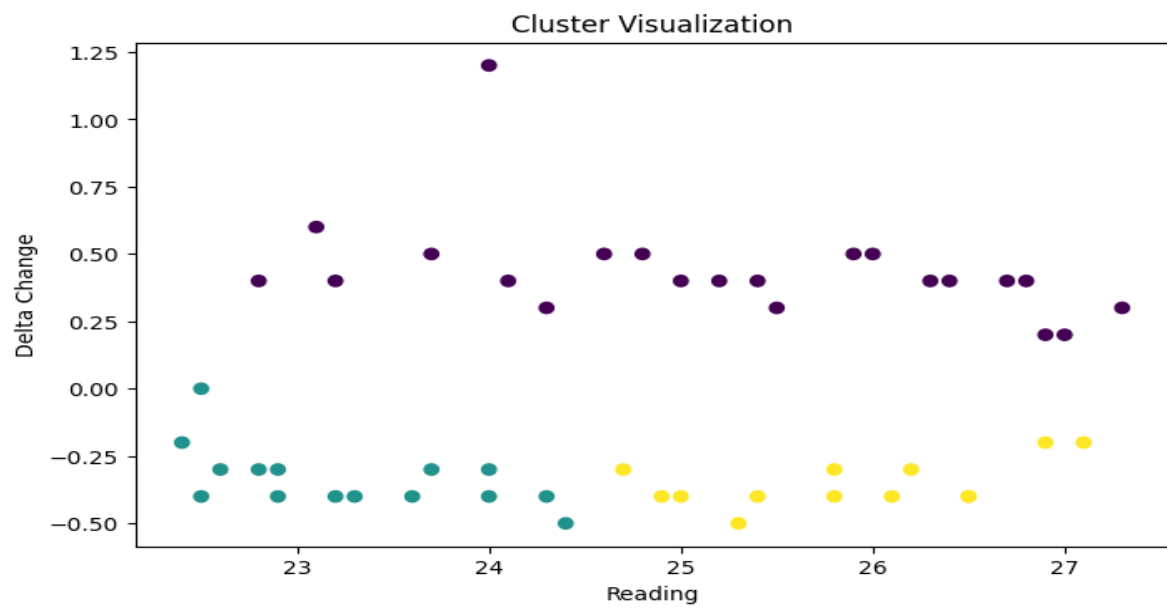
■ Output:

```
reading Delta_Change
0 22.5 0.0
1 23.1 0.6
2 22.8 -0.3
3 24.0 1.2
4 24.3 0.3
```



```
Clustered Data:
  reading Delta_Change Inherent_Cluster
0 22.5 0.0 1
1 23.1 0.6 0
2 22.8 -0.3 1
3 24.0 1.2 0
4 24.3 0.3 0

Silhouette Score:
0.4808157926191231
```



Practical 28 — Lab Mining Data – Machine Learning Model Pipeline

■ Program Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import joblib

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# STEP 1: LOAD DATA

df = pd.read_csv('lab_mining_data.csv')
print(df.head())

# STEP 2: CORRELATION ANALYSIS

corr_matrix = df.corr()

plt.figure(figsize=(10,8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm'
)

plt.title('Correlation Heatmap')

plt.show()

# STEP 3: CLUSTERING

scaler = StandardScaler()

X_scaled = scaler.fit_transform(
    df[['reading', 'Delta_Change']]
)

kmeans = KMeans(
    n_clusters=3,
    init='k-means++',
    n_init=10,
    random_state=42
)
```

```
df['Inherent_Group'] = kmeans.fit_predict(
    X_scaled
)

print("\nClusters Created")

# STEP 4: ANOMALY CREATION

threshold = (
    df['reading'].mean() +
    2 * df['reading'].std()
)

df['Anomaly'] = (
    df['reading'] > threshold
).astype(int)

# STEP 5: CLASSIFICATION

X = df[
    [
        'reading',
        'Delta_Change',
        'Inherent_Group'
    ]
]

y = df['Anomaly']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

clf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

clf.fit(X_train, y_train)

# STEP 6: PREDICTION

y_pred = clf.predict(X_test)

print("\nClassification Report")

print(
    classification_report(
```

```

        y_test,
        y_pred
    )
)

# STEP 7: SAVE MODEL

joblib.dump(
    clf,
    'lab_mining_model.pkl'
)

print("\nModel Saved Successfully")

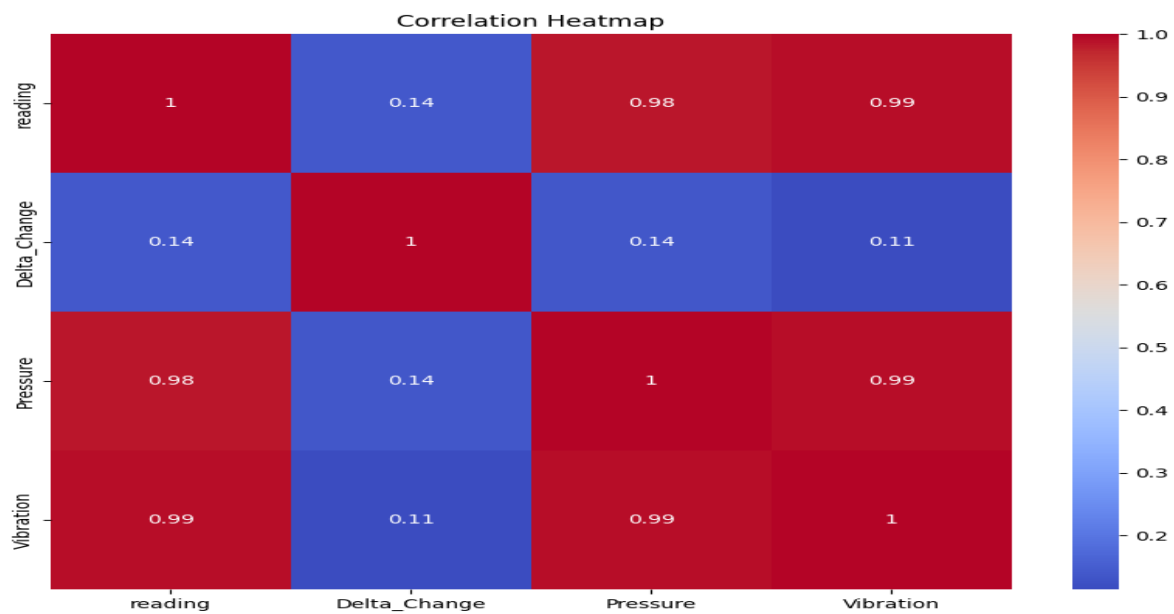
```

■ Output:

```

reading Delta_Change Pressure Vibration
0 22.5 0.0 101 0.20
1 23.1 0.6 102 0.22
2 22.8 -0.3 100 0.21
3 24.0 1.2 104 0.25
4 24.3 0.3 105 0.26

```



Clusters Created

```

Classification Report
      precision recall f1-score support

0 1.00 1.00 1.00 10

accuracy 1.00 10
macro avg 1.00 1.00 1.00 10
weighted avg 1.00 1.00 1.00 10

Model Saved Successfully

```